

Singapore Polytechnic

School of EEE

IC4004_IC4003_IC4001 Internship

Written Assignment

Diploma in Computer Engineering

Semester 2, 2025/26

Prepared For: Mr Andrew Tan, Mr Ng Yong Thiang

Lecturer Name

Prepared By: Er Zhong Xun Ervin

Student Name

Class: DCPE/FT/3B/04

Table Of Contents

Table Of Contents.....	2
1.0 Introduction.....	5
1.1 Name, History & Background of the Company.....	5
1.2 Brief Details on Internship.....	6
1.3 Context.....	7
1.4 Scope.....	7
2.0 Work Experience & Journey.....	8
2.1 Conceive.....	8
2.1.1 Problem Statement.....	8
2.1.2 Goals.....	8
2.2 Design.....	9
2.2.1 Existing Solutions.....	9
2.2.2 Sketch.....	9
2.2.3 Technology Used.....	10
2.2.4 System Architecture.....	11
2.2.5 Data Flow.....	12
2.2.6 Features & Functions.....	13
2.3 Implementation.....	14
2.3.1 Locally Hosted.....	14
2.3.2 Backend.....	14
Retrieval Augmented Generation (RAG).....	14
Sub ATA Code Extraction.....	17
AI Summarization/Recommendation.....	18
Damage Assessment for Defects powered by Image Search.....	19
2.3.3 Frontend.....	21
Web Application/Search Function.....	21
Chat History.....	23
Audit Log.....	23

File Upload.....	24
2.3.4 Multiple Machine Accessibility.....	25
2.3.5 Other Additional Features.....	25
Streaming via FlaskAPI.....	25
Temporary Saving of Sub ATA Code List.....	26
Direct Search.....	26
2.4 Operation.....	27
3.0 Challenges Faced & Problem Solving.....	28
4.0 Outcome.....	29
4.1 Avotron Pte. Ltd.....	29
4.2 5G&AIOT Centre.....	29
4.3 Team Collaboration.....	29
5.0 Impact.....	30
5.1 Avotron Pte. Ltd.....	30
5.2 5G&AIoT Centre.....	30
5.3 Other Fields/Industry Applications.....	30
6.0 Key Skills.....	31
6.1 Backend (Programming, Experimenting, Prototyping).....	31
6.2 Frontend (Web Development).....	31
7.0 Conclusion.....	32
8.0 Additional Project - Smart AI Advertiser.....	33
8.1 Scope.....	33
8.2 Purpose.....	33
8.3 Problem Statement.....	33
8.4 Sketch & Solution.....	34
8.5 Data Flow.....	35
8.6 System Architecture.....	35
8.7 My Contribution.....	36
8.8 Contribution Features.....	36

8.9 Pictures.....	37
9.0 References & Learning Resources.....	38
9.1 Learning Resources & Documentation.....	38
9.2 Backend Developement.....	38
9.3 AI Models.....	38
9.4 Development Tools & Installation Guide.....	38
9.5 Additional Tools.....	38
10.0 Appendix.....	39
10.1 Deployment Instructions.....	39
10.1.1 Ollama.....	39
10.1.2 Docker.....	39
10.1.3 Python.....	41
10.1.4 Visual Studio Code.....	41
10.1.5 Launching Web Application.....	44
10.2 Web Application Instructions.....	45
10.2.1 Login.....	45
10.2.2 Search.....	45
10.2.3 Sub ATA Code Retrieval/Extraction.....	46
10.2.4 History.....	47
10.2.5 Defect Assessment via Image Search and Location Selection.....	48
10.2.6 Temporary Saving of Selected ATA Sub-Codes.....	49
10.2.7 Audit Log.....	50
10.2.8 Logout.....	50
10.2.9 File Upload.....	50
10.3 Cosine Similarity.....	51

1.0 Introduction

1.1 Name, History & Background of the Company

Avotron Pte. Ltd. an exempt Private Company Limited by Shares, incorporated on 14 August 2008 in Singapore. The company’s main activity is to manufacture and repair engineering and scientific instruments while the secondary activity is to perform wholesale trade of a variety of goods without a dominant product.

Over the years, Avotron Pte. Ltd. has successfully undertaken government procurement projects, notably with Singapore Polytechnic. (Refer to Fig. 1.1)

- In September 2024, the company delivered the *Supply, Installation, Testing and Commissioning for Asset Management System*.
- In February 2025, it completed the *Supply, Delivery, Installation, and Commissioning of the Aircraft Cockpit Trainer for the Aircraft Instrument System Module*.

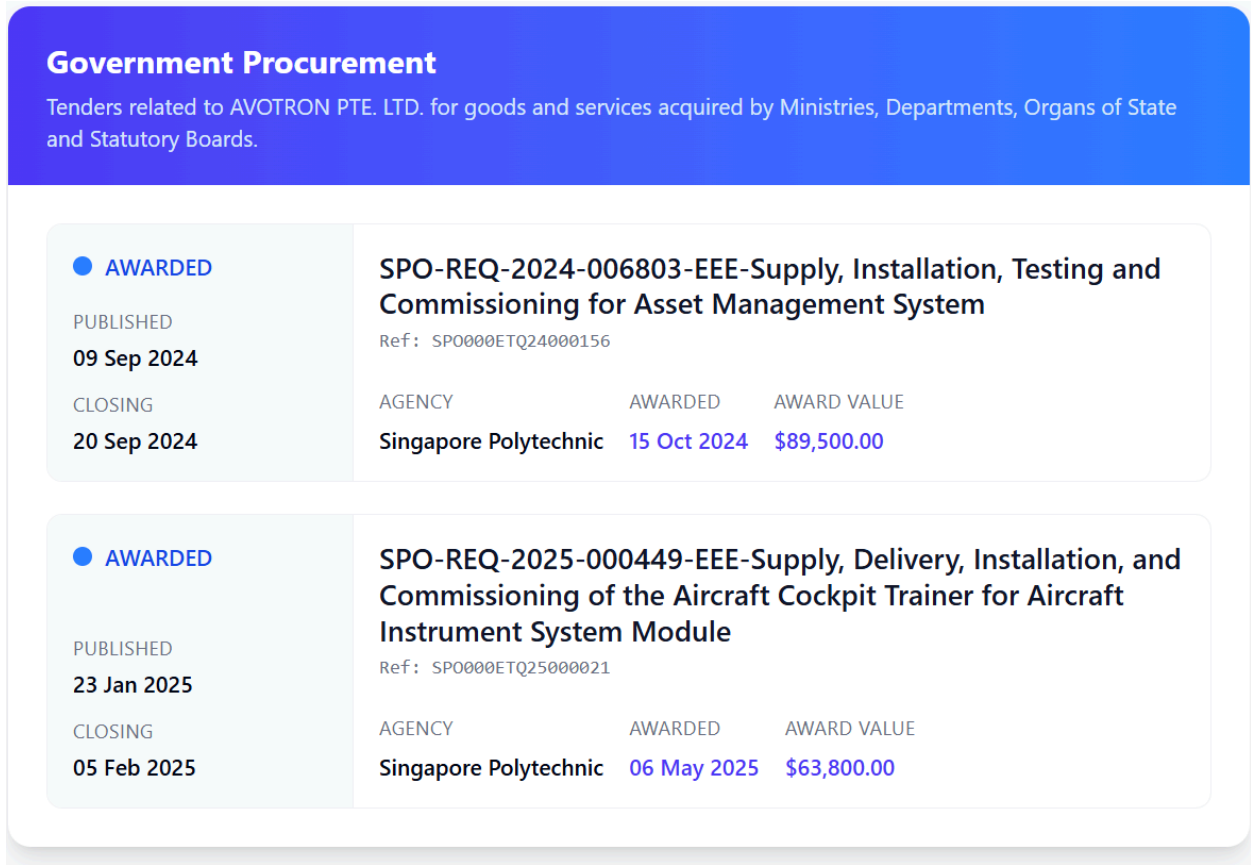


Fig. 1.1: Government Procurement provided by recordowl.com

1.2 Brief Details on Internship

The internship was carried out at the 5G & AIoT Centre in Singapore Polytechnic (SP). Although the project was primarily a solo assignment, the environment provided valuable opportunities to interact with other interns and staff within the centre. I also had the opportunity to participate in events or tours organised by SP.

Working alongside Burmese colleagues fostered a collaborative and inclusive atmosphere. We talked and shared about our food, culture, and language, which helped build strong friendships and mutual respect.



Picture with fellow interns in 5G&AIoT Centre



Participation at SP Industry & Innovation (I&I)

1.3 Context

In the aviation industry, aircraft maintenance specialists rely heavily on vast and complex technical manuals to carry out inspections, troubleshooting, and component replacements. However, locating the exact manual and relevant sections corresponding to a specific system or component can be time-consuming and inefficient.

This project aims to develop an intelligent, user-friendly platform that streamlines the retrieval of aircraft maintenance documentation. This will allow maintenance personnel to search for systems or components by name, part number, ATA chapter, or function. It will then intelligently recommend and link to the appropriate manual(s) and highlight specific sections relevant to the query.

By incorporating a structured data model, keyword tagging, and AI-powered content, the tool will significantly reduce manual lookup time, minimize errors, and enhance maintenance productivity. The solution will also support future scalability to integrate various aircraft models and maintenance documentation formats.

This project is expected to bridge the gap between complex technical documentation and practical on-ground maintenance tasks, supporting faster turnaround and improved operational safety.

1.4 Scope

The project is to design and develop a web-based dashboard for an **Aircraft Documentation Platform Using Generative AI for Maintenance Manual Access** for aircraft maintenance specialists with the following features:

- System architecture of the dashboard.
- Tagging/indexing to relate systems and components with relevant manual sections.
- Documentation references to ATA chapters or component codes.
- A smart search function that accepts queries by Component/System name, ATA chapter, part number or fault code.

2.0 Work Experience & Journey

2.1 Conceive

The project begins with a clear focus on understanding the needs of aircraft engineering specialists. They require a solution to quickly find, understand, and act on the right information from dense aviation manuals. Since these manuals are classified, the system must be locally hosted to ensure security and confidentiality.

To address these requirements, the project envisions a scalable Retrieval-Augmented Generation (RAG) pipeline. This pipeline is designed to integrate multiple types of aviation manuals, both current and future, ensuring adaptability and long-term usability.

2.1.1 Problem Statement

Aircraft maintenance specialists often face significant inefficiencies when searching for relevant information within extensive technical manuals. The sheer volume and complexity of these documents can lead to delays in locating the required data, increasing workload and reducing productivity. More critically, these delays pose potential risks to operational safety, as timely and accurate access to information is essential for effective maintenance and repair activities.

2.1.2 Goals

- **Improve Information Retrieval Efficiency:** Reduce the time required to locate critical data within extensive manuals.
- **Enhance Operational Safety:** Ensure timely and accurate access to information to minimize risks during maintenance and repair activities.
- **Support User Productivity:** Provide intuitive tools that streamline workflows and reduce cognitive load.
- **Maintain Data Security:** Safeguard sensitive aviation manuals and operational data through secure hosting and audit mechanisms.
- **Enable Scalability and Flexibility:** Ensure the system can adapt to multiple machines, larger datasets, and evolving maintenance needs.

2.2 Design

2.2.1 Existing Solutions

1. SOMA Software: Centralized dashboard for all aircraft documents (manuals, maintenance records, certifications).

- Automated expiration tracking for compliance.
- Real-time visibility into document status.
- Aircraft alerts & notifications

However, it's primarily a compliance and tracking tool and storage, not designed for deep parsing. No usage of Gen AI to help with user experience.

2. Microsoft 365: AI powered search/finding of documents called Copilot Search (Microsoft's Gen AI).

- Automated expiration tracking for compliance.
- Real-time visibility into document status.
- Aircraft alerts & notifications

Although optimized for finding documents, it's not extracting specific procedures often requiring multiple prompts and lacking domain-specific accuracy.

2.2.2 Sketch

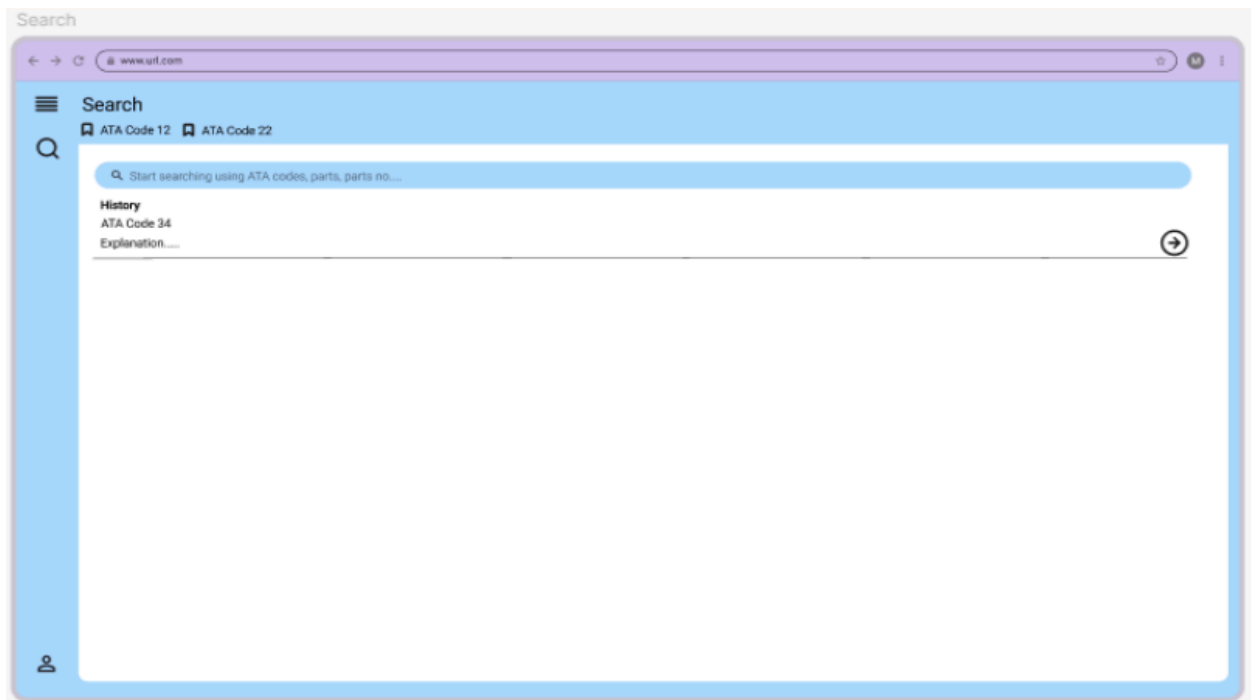


Fig. 2.1: Sketch of Web Application

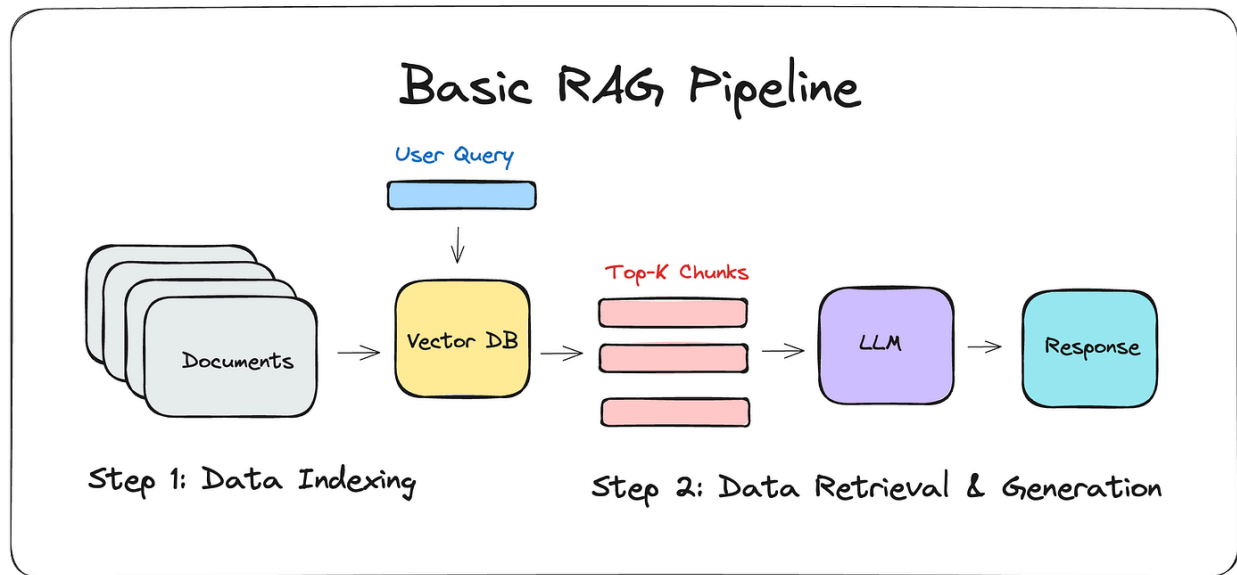


Fig. 2.2: Simple RAG Pipeline Sketch by medium.com

Fig. 2.1 provides me with an early visualization of how the web application should be designed. It shows the main Search function with a side bar on the left to show the user's history. This sketch guided the initial design direction and served as a reference before moving into detailed prototyping.

While Fig. 2.2 provides me with a basic idea of what technology a simple RAG Pipeline requires, whether it would be for the vector database, large language model or even the embedding model for the metadata.

2.2.3 Technology Used

All Open Source & Locally Hosted:

- **Python** - Backend Development
- **HTML/CSS/JS** - Frontend Development
- **Hugging face all-MiniLM-L6-v2** - Embedding Model
- **Ollama phi4:14b** - Large Language Model (LLM)
- **Ollama qwen2.5-vl-7b** - Vision Language Model (VLM)
- **Qdrant** - Vector Database
- **MinIO** - Blob Storage (For Files)
- **DuckDB** - SQL OLAP database management system (For Website data)

2.2.4 System Architecture

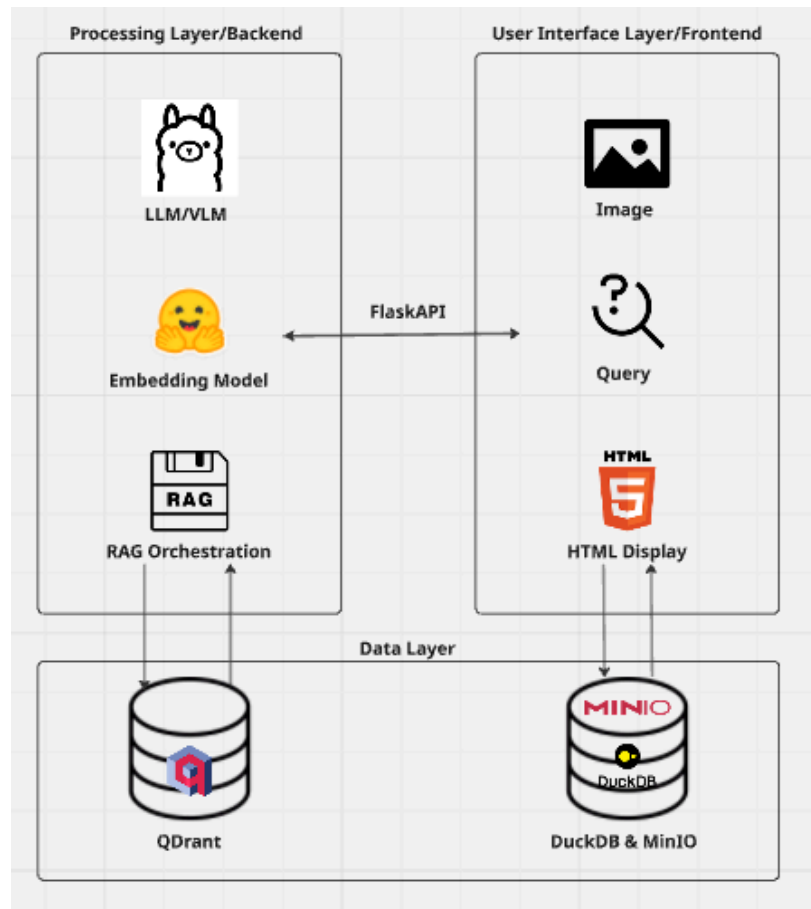


Fig. 3.1: System Architecture

The system architecture consists of three main layers: the processing layer in Python for backend, connected to the user interface layer in HTML/CSS/JS for frontend through FlaskAPI and lastly, a data layer containing the three different databases for all our data management. (Refer to Fig. 3.1)

- **User Interface Layer:**
Captures user input and sends it to the backend through FlaskAPI. This layer accepts two inputs: Image & Text in natural language queries. Lastly, it also renders the output from backend processing for users to look at.
- **Processing Layer:**
Handles core logic, orchestration, and model inference. This layer uses three pre-trained locally hosted models:
 - **Ollama phi4:14b** - Large Language Model (LLM)
 - **Ollama qwen2.5-vl-7b** - Vision Language Model (VLM)
 - **Huggingface all-MiniLM-L6-v2** - Embedding Model

Results of the backend processing are then sent back to the user interface layer through FlaskAPI for display

- **Data Layer:**

Stores and retrieves structured and unstructured data. This layer contains three different types of database:

- **Qdrant** - Vector database for embedding-based storage and retrieval.
- **MinIO** - Blob storage for large aviation manuals.
- **DuckDB** - Lightweight analytical database for multi-accounts with their individual structured queries and history.

Qdrant connects to the Processing Layer, whereas DuckDB and MinIO are integrated with the User Interface Layer.

2.2.5 Data Flow

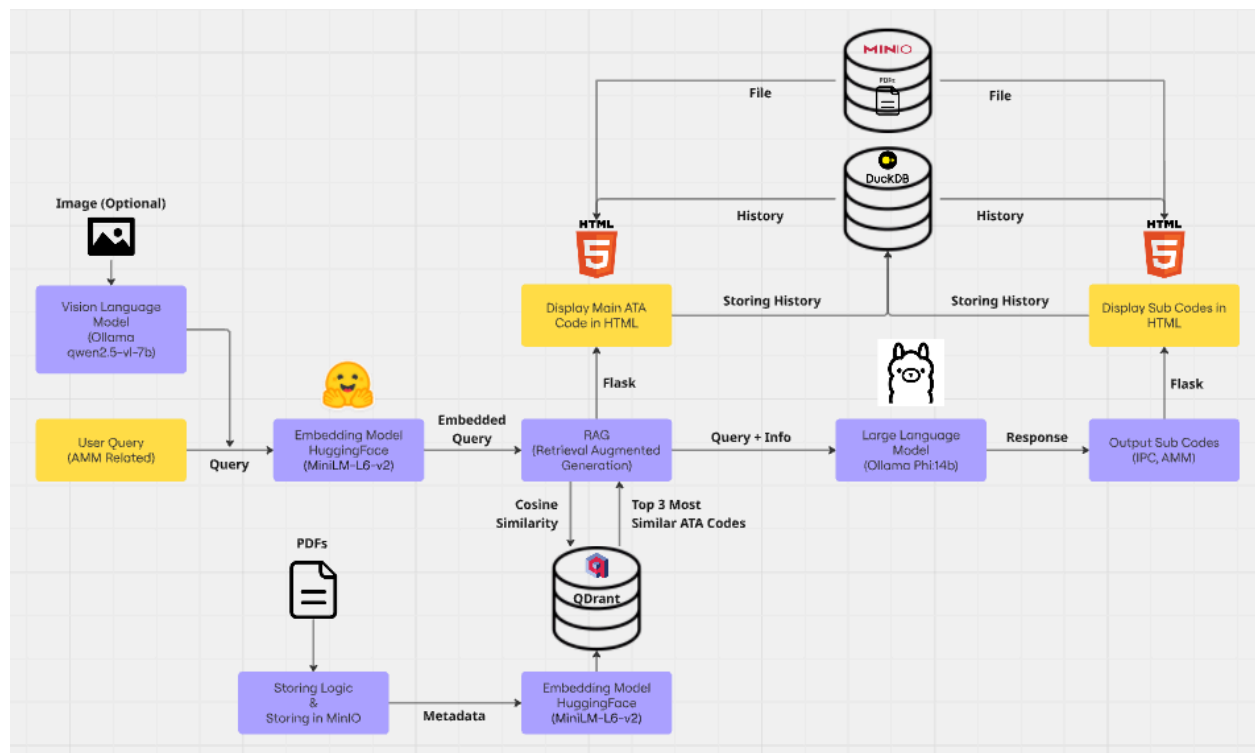


Fig. 4.1: Data Flow

This data flow diagram illustrates the pipeline designed to process and display ATA (Air Transport Association) codes and subcodes from aircraft maintenance documentation. (Refer to Fig. 4.1)

The data flow provides a clear understanding of how information moves through each stage of the pipeline:

1. A user can initiate a query using either an image or text. If an image is uploaded, it is processed by the vision-language model, which converts it into a text-based query.
2. The query is then passed to the embedding model, which enables the Retrieval-Augmented Generation (RAG) process. Using cosine similarity, the vector database (Qdrant) retrieves the top K most relevant results.
3. These results are sent from the backend to the frontend via FlaskAPI and displayed in the HTML web application.
4. When a user opens and reviews the retrieved manual information files, the corresponding files are fetched from MinIO for viewing.
5. If the user requests sub-code extraction, the query and manual content are processed by the large language model, which generates the extracted sub-codes.
6. The outputs are again transmitted through FlaskAPI and rendered in the HTML interface.
7. Application data such as chat history, sessions, and queries are logged and stored in DuckDB for future reference.
8. Finally, users have the option to upload additional files or manuals into MinIO and Qdrant, ensuring they are available for storage and retrieval within the system.

2.2.6 Features & Functions

1. Smart Search Function (Ensure users can obtain the right information quickly)

- Image/Natural Language Search for RAG
- AI Summary/Suggestions
- Quick Retrieval of Manuals
- Code Extraction using LLM
- Web Application
- Chat History

2. Secure (Ensure sensitive information such as aviation manuals are kept safe)

- Audit Log
- Locally Hosted

3. Scalable & Others (Features that are added to make the user experience better)

- Multiple Machine Accessibility/Multiple Accounts
- Damage Assessment for Defects powered by Image Search
- File Uploading for Scalability

2.3 Implementation

2.3.1 Locally Hosted

For security purposes, all components are locally hosted. Qdrant and MinIO are containerized using Docker, Ollama is installed to host LLMs and VLMs, the embedding model is stored on the local machine, and DuckDB is integrated via its Python library for local data management.

2.3.2 Backend

Retrieval Augmented Generation (RAG)

To perform natural language search using RAG, we first need to pre-process and store the metadata in a vector database (Qdrant).

```
page_num = int(page_match.group(1)) if page_match else fallback_page_num
page_pdf = fallback_page_num

date = date_match.group(0) if date_match else None

return {
    "file_name": file_name,
    "file_path": object_name,
    "headers": header,
    "functions": json.dumps(functions) if functions else None,
    "sequential_numbers": json.dumps(sequential_numbers) if sequential_numbers else None,
    "tasks": json.dumps(tasks_id) if tasks_id else None,
    "task_codes": json.dumps(task_codes) if task_codes else None,
    "chapter": chapter,
    "section": section,
    "subject": subject,
    "ata_code": ata_code,
    "main_chapter": main_chapter_code,
    #manualpage
    "pagenumber": page_num,
    #pdfpage
    "pagepdf": page_pdf,
    "page_code": page_code,
    "page_code_count": final_count,
    "date": date,
    "first": is_first,
    "chunk_text": chunk_text,
    "indicator": indicator
}
```

Fig. 5.1: Metadata for Aircraft Maintenance Manual (AMM)

```

points = []
if header:
    points.append(PointStruct(
        id=str(uuid4()),
        vector=embedding_function.encode(header),
        payload==metadata
    ))

for chunk in chunks:
    points.append(PointStruct(
        id=str(uuid4()),
        vector=embedding_function.encode([chunk])[0],
        payload=metadata
    ))

client.upsert(
    collection_name=COLLECTION_NAME,
    wait=True,
    points=points
)

```

Fig. 5.2: Payload & Storing

Headers, ATA codes, and page numbers are extracted from the PDF using the PyPDF2 library and regular expressions (regex). These elements are stored as metadata for future retrieval through AI, specifically embedding models, which capture semantic meaning and contextual relationships. The metadata is then integrated into the payload, with the corresponding header and chunked text tagged to ensure organized storage and efficient indexing. (Refer Fig. 5.1 & 5.2)

```

SRM_first_result = client.query_points(
    collection_name=COLLECTION_NAME,
    query=query_vector,
    limit=3,
    with_payload=True,
    score_threshold=0.2,
    query_filter=Filter(
        must=[
            FieldCondition(key="indicator", match=MatchValue(value="SRM")),
            FieldCondition(key="first", match=MatchValue(value=1))
        ])
    )

for point in AMM_first_result.points:
    responsetoquestion = ""
    page_chunks = []
    first_payload = point.payload or {}
    page_code = first_payload.get("page_code", 'Unknown')

    if page_code in seen_page_codes:
        continue
    seen_page_codes.add(page_code)

    newfilename = first_payload.get('file_name', 'Unknown')
    newpageNumber = first_payload.get('pagenumber', 'Unknown')
    filepath = first_payload.get('file_path', 'Unknown')
    header = first_payload.get('headers', 'Unknown')
    tasks = first_payload.get('task_codes', 'Unknown')
    chunk_text = first_payload.get("chunk_text", 'Unknown')
    first_page_pdf = first_payload.get("pagepdf", "Unknown")
    final_page_count = first_payload.get("page_code_count", 'Unknown')
    main_chapter = first_payload.get("main_chapter", "Unknown")

```

Fig. 5.3: Retrieval Logic

```

code_entry = {
    "code": code,
    "description": line_without_page,
    "filepath": filepath,
    "page": pdf_page,
    "source_page": page_num
}
sub_code_entries.append(code_entry)

yield f"data: {json.dumps({'new_code': code_entry, 'main_atc_code': main_atc_code})}\n\n"

progress = base_progress + (result_idx * progress_per_result) + \
    ((idx + 1) / total_pages) * progress_per_result
yield send_progress(int(progress), f"Extracting codes from page {page_num} ({idx+1}/{total_pages})")

```

Fig. 5.4: Stream results via FlaskAPI

For retrieval, the system uses cosine similarity to identify the top-K results along with their associated metadata. Cosine similarity measures the angle between two non-zero vectors in an inner product space, providing a metric of their closeness. (Refer to Annex 9.3)

The corresponding payloads are then returned together with the previously stored metadata. (Refer to Fig. 5.3)

These results are then streamed to the frontend for display, enabling users to view both the retrieved content and its contextual information in real time. (Refer to Fig. 5.4)

Sub ATA Code Extraction

For the sub-ATA code extraction function, the system leverages AI through LLMs. By providing chunked text from the relevant manual pages, the LLM extracts and returns the corresponding ATA codes.

```
def extract_codes_from_single_page(page_text, main_ata_code, page_number):
    prompt = f"""
    You are an assistant that extracts maintenance codes from aircraft maintenance manuals.

    Text from page {page_number}:
    {page_text}

    Extract all the codes under the "References" section.

    Extract ALL codes ONLY in this format:
    - ATA Codes: AMM/SRM DD-DD-DD (e.g. AMM 05-41-00, SRM 32-21-01, 46-02-10)
    - IPC Codes: IPC DD-DD-DD(-DDD) (e.g. IPC 34-05-12, IPC 11-11-00-37B, IPC 11-11-00-37)
    EXCLUDE:
    - {main_ata_code}
    - Steps, Tasks (e.g. S 862-070, S 062-075)

    For each code found, provide brief description (max 8 words)

    Respond with ONLY a valid JSON array of strings. No markdown, no explanations.
    This is an example only, DO NOT INCLUDE THESE in your response: ["12-40-04/201,Bird Strike Cleaning, SRM 27-81-11,Leading Edge Flaps.
    """

    try:
        response = ollama.generate(
            model=OLLAMA_MODEL,
            prompt=prompt,
            options={
                "temperature": 0.0,
                "top_p": 0.9,
                "num_predict": 500
            }
        )
```

Fig. 5.5: Prompt & Settings for Sub ATA Code Extraction (LLM)

In Fig. 5.5, the prompt is designed to capture only Aircraft Maintenance/Structural Repair Manuals (AMM/SRM) and Illustrated Parts Catalog (IPC) ATA codes. The system responds in JSON (JavaScript Object Notation) format, enabling easier parsing, integration, and downstream processing.

To improve the accuracy of sub-code extraction, several parameters are configured:

- temperature is set to 0 to eliminate randomness,
- top_p is set to 0.9 to provide additional descriptive context for each ATA code
- num_predict is set to 500 to support the extraction of multiple ATA codes simultaneously.

Instead of inserting large text chunks from all relevant documents into the LLM for extraction, I chose to process the content page by page. Although this approach takes more time, it ensures higher accuracy and reduces the risk of hallucinations or the model experiencing “Lost in the Middle” which is to lose context in the middle of long inputs and only register the top and the bottom of the input.

AI Summarization/Recommendation

For the AI Summarisation/Recommendation function, the system leverages AI through LLMs, configured with similar parameters as in Fig. 5.5.

```
prompt = f"""
Help summarize this chunk text into a short description less than 40 words using the context given for viewing.
Just give the information needed in your response, NOTHING else.

Context: {header}

Text: {chunk_text}
"""
```

Fig. 5.6: Prompt for Summarising ATA Sub Code Chapters (LLM)

In Fig. 5.6, the prompt is designed to summarise the body text of the extracted sub-ATA code manuals (AMM), using the chapter header as contextual guidance, and constraining the output to fewer than 40 words. By supplying chunked text from the relevant manual pages, the LLM processes the input and returns concise summaries of each ATA sub-code.

```
prompt = f"""
You are an aviation maintenance assistant. Based on these recent ATA code searches: "${historyContext}"

Generate 3-8 SHORT related search suggestions for aircraft systems, ATA codes, maintenance, or parts. Each suggestion should be 2-4 words maximum.
For ATA codes, use the format "XX-XX-XX" (e.g. 10-11-20) AND include at least one code suggestion if relevant.

Rules:
- Only output the suggestions, one per line
- No numbering, bullets, or explanations
- Focus on related aircraft systems or maintenance tasks
- Keep suggestions SHORT and specific
- Use aviation/ATA terminology from AMM manuals

Examples of good suggestions:
Engine Oil System
10-11-20
Fuel Pump Check
Bird Strike Inspection

Your suggestions:
"""
```

Fig. 5.7: Prompt for AI Recommendations (LLM)

In Fig. 5.7, the prompt is designed to generate query recommendations by leveraging the current user's history as contextual guidance. The output is constrained to between 2 and 4 words per suggestion, with a requirement to produce 3 to 8 recommendations in total. Additionally, if relevant, one of the suggestions must include an ATA code.

Damage Assessment for Defects powered by Image Search

For this feature, the system leverages AI through VLMs, configured with similar parameters as in Fig. 5.5.



Fig. 5.8: Open-Source Roboflow Object Detection Computer Vision Model for Aircraft Defects

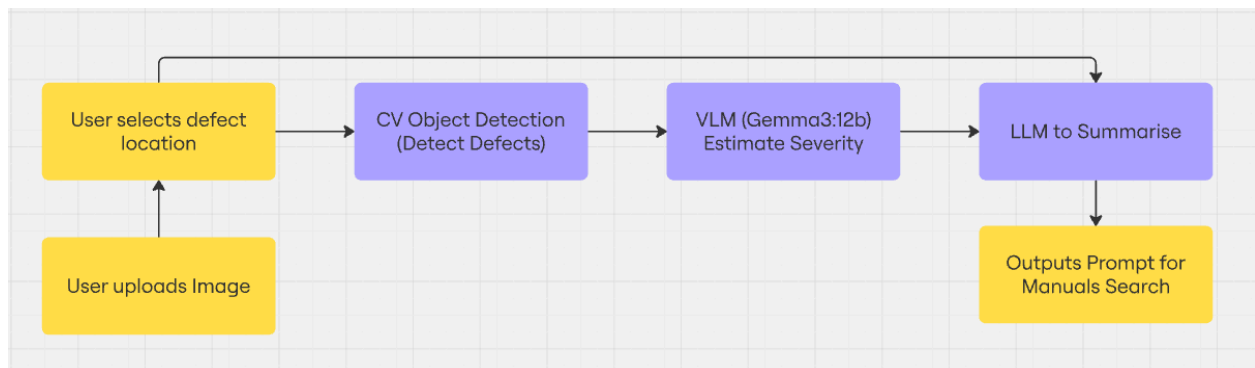


Fig. 5.9: Original Workflow for Damage Assessment Feature

Fig. 5.9 illustrates the original workflow for the damage assessment feature. In the initial design, an object detection computer vision model was employed to detect and classify defects. (Refer to Fig. 5.8) The relevant information was then passed into a VLM for severity assessment, followed by a LLM for summarisation and display.

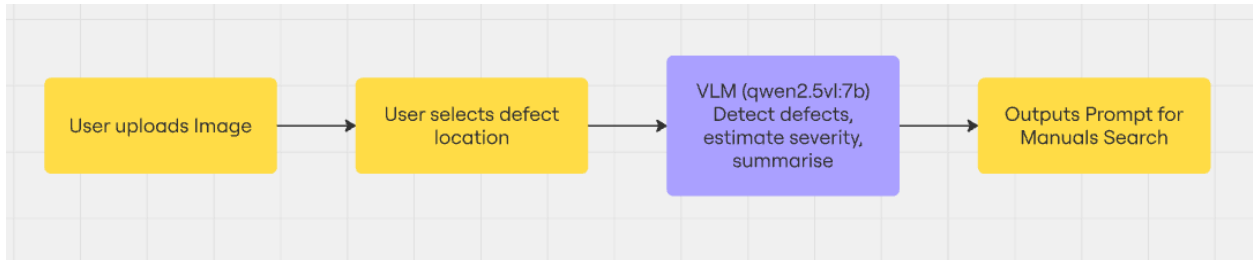


Fig. 5.10: Current Workflow for Damage Assessment Feature

Defects: Scratch, Low Severity

Fig. 5.11: Response by Initial Workflow (Fig. 5.9)

Defects: Paint-Off, Dent, Low Severity

Fig. 5.12: Response by Current Workflow (Fig. 5.10)

However, I realised that instead of combining a computer vision object detection model and LLM with a VLM, relying solely on a VLM achieves equal or better results in less time seen in Fig. 5.10. This approach also enables classification across a wider range of defects, since traditional object detection models are constrained by the limited number of defect categories they are trained on. (Refer to Fig. 5.11 & Fig. 5.12)

2.3.3 Frontend

Web Application/Search Function

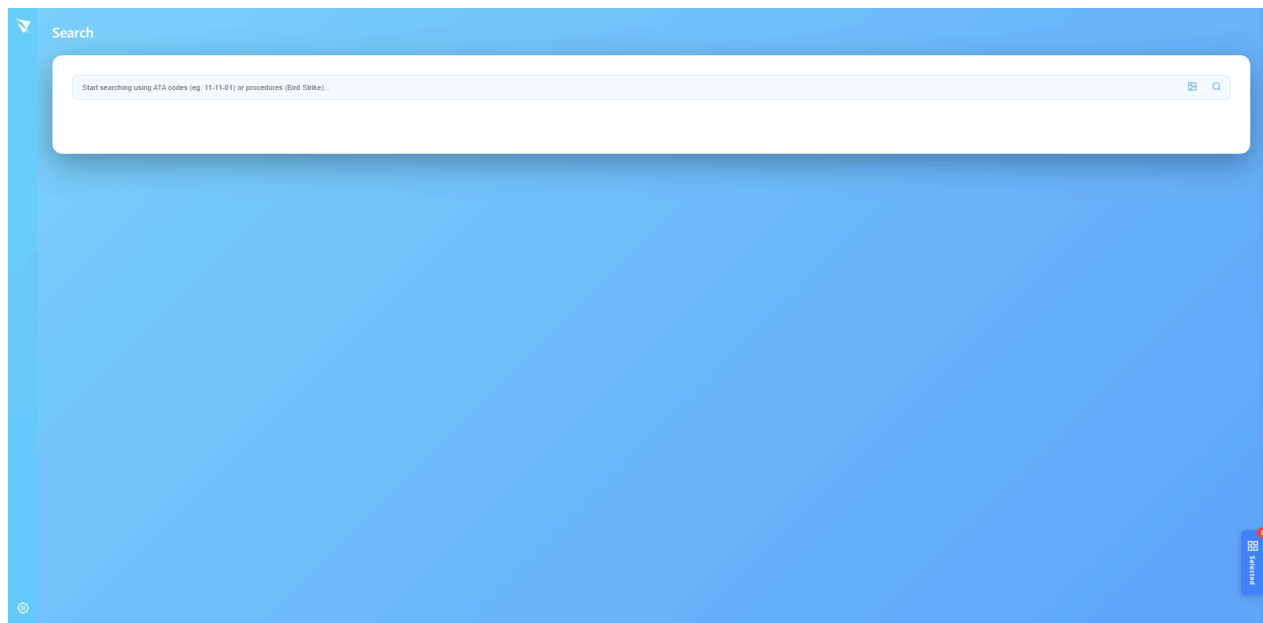


Fig 6.1: Main Page

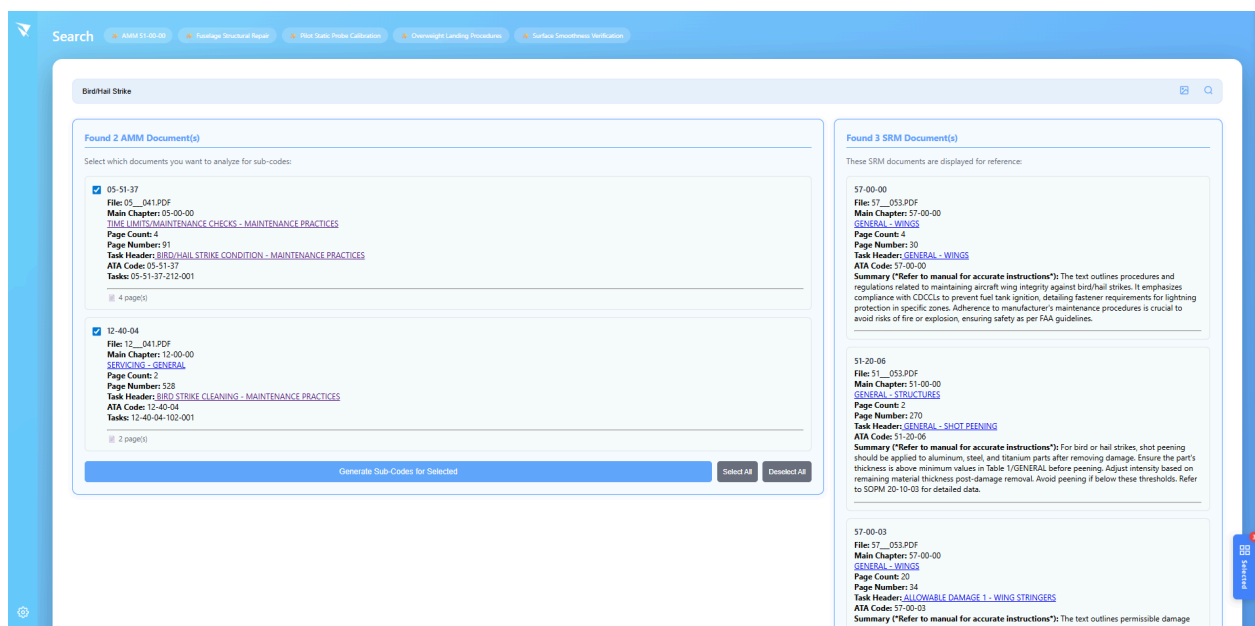


Fig. 6.2: Selection of Manuals (Retrieval)

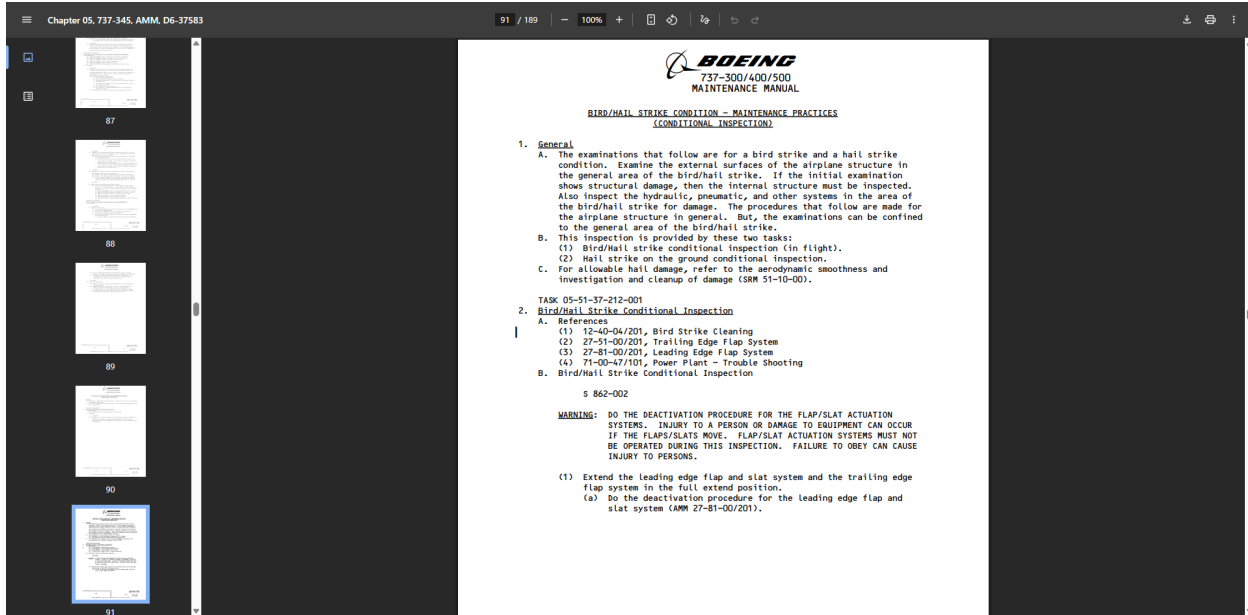


Fig. 6.3: Validation (Manual)

```
f"<p><b>Header:</b><a href='http://{DESKTOP}:9000/aircraft/{filepath}#page={first_page_pdf}' target='_blank'> {header}</a></p>"
```

Fig. 6.4: HTML Snippet for clickable link to Manual PDF

In Fig. 6.1, the starting page of the web application is displayed. The user can click on the search bar and type natural language queries related to the manual or maintenance procedures. Above the search bar, AI-generated query suggestions are provided based on the user's history, allowing quick selection and search.

Examples of queries include:

- "How to fix bird strike?" -> AMM/SRM
- "AMM How to fix bird strike?" -> AMM Only (Instant Search for Validation)
- "AMM XX-XX-XX" -> AMM Direct Chapter (Instant Search for Extraction)
- "IPC XX-XX-XX-XXX" -> IPC Direct Chapter (Instant Search)
- "SRM XX-XX-XX" -> SRM Direct Chapter

In Fig. 6.2, the files, headers, and pages of the ATA code manual, along with the main chapter, are made accessible to the user for validation after performing a search.

Fig. 6.3 illustrates the opened manual PDF that the user can use to validate the retrieved information. The PDF is retrieved directly from the blob storage, which contains all the documents, and is accessed through a hyperlink generated by an HTML snippet that connects to the MinIO database. (Refer to Fig. 6.4)

Chat History

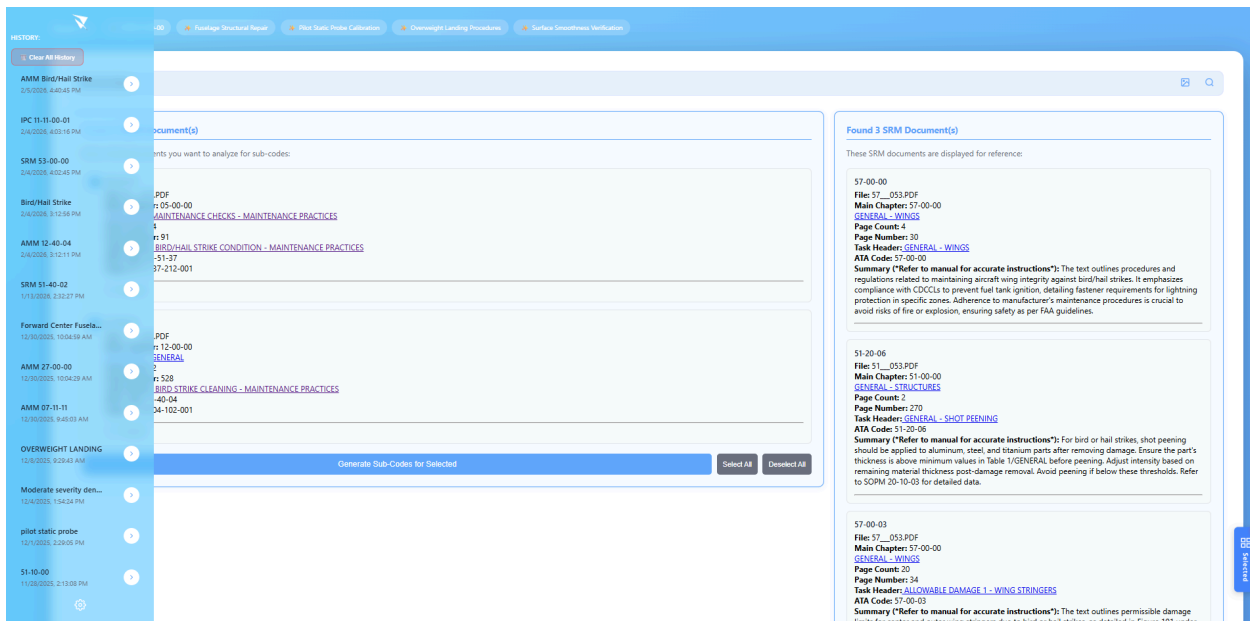


Fig. 6.5: Chat History

Fig. 6.5 illustrates the chat history, which can be accessed by hovering over the left side of the page to open the history panel. The feature stores the user's previous searches, displaying each query along with the corresponding date and time it was performed.

Each account is assigned a unique DuckDB ID. Every query and extraction is stored in DuckDB in a structured format, together with the corresponding date and time at which it was performed.

Audit Log

Audit Log				Logout
Time	Task Name	Task Description	User	
2/10/2026, 8:58:29 AM	Preview Search	Bird/Hail Strike	admin	
2/10/2026, 8:57:48 AM	Login	-	admin	
2/5/2026, 4:40:45 PM	Search	AMM Bird/Hail Strike	admin	
2/4/2026, 4:02:57 PM	Preview Search	Perform INSPECTION AND REMOVAL OF DAMAGE, Crack, Section D - Air Center Fuselage, STA 794.3, High severity	admin	
2/4/2026, 4:03:56 PM	Image Search	-	admin	
2/4/2026, 4:03:16 PM	Search	IPC 11-11-00-01	admin	
2/4/2026, 4:02:45 PM	Search	SRM 53-00-00	admin	
2/4/2026, 4:00:45 PM	Preview Search	Bird/Hail Strike	admin	
2/4/2026, 4:00:12 PM	Login	-	admin	
2/4/2026, 3:14:30 PM	Logout	-	admin	
2/4/2026, 3:13:33 PM	Preview Search	Perform INSPECTION AND REMOVAL OF DAMAGE, Hole, Section C - Center Fuselage, STA 685, High severity	admin	
2/4/2026, 3:13:32 PM	Image Search	-	admin	
2/4/2026, 3:12:56 PM	Search	Bird/Hail Strike	admin	
2/4/2026, 3:12:11 PM	Search	AMM 12-40-04	admin	
2/4/2026, 3:11:50 PM	Login	-	admin	
2/2/2026, 11:54:41 AM	Login	-	admin	
1/21/2026, 2:13:50 PM	Login	-	admin	
1/21/2026, 12:48:44 PM	Preview Search	Perform INSPECTION AND REMOVAL OF DAMAGE, Scratch, Paint Off, Low severity	admin	
1/21/2026, 12:48:44 PM	Image Search	-	admin	
1/21/2026, 12:41:31 PM	Login	-	admin	
1/21/2026, 12:39:41 PM	Logout	-	admin	
1/21/2026, 12:18:50 PM	Login	-	admin	
1/19/2026, 12:45:02 PM	Login	-	admin	
1/19/2026, 11:58:29 AM	Login	-	admin	

Fig. 6.6: Audit Log

Fig. 6.6 illustrates the audit log, which records the user's activities by tracking the task name, task description, and the exact date and time each action occurred. Currently, it tracks:

- Login
- Logout
- Preview Search (Validation)
- Image Search (Damage Assessment)
- Full Search (Extraction)

Each log is also stored under the user's unique DuckDB ID in a structured format, together with the task description and the corresponding date and time at which it was performed. This feature ensures transparency and allows users to review their actions for accountability and validation.

File Upload

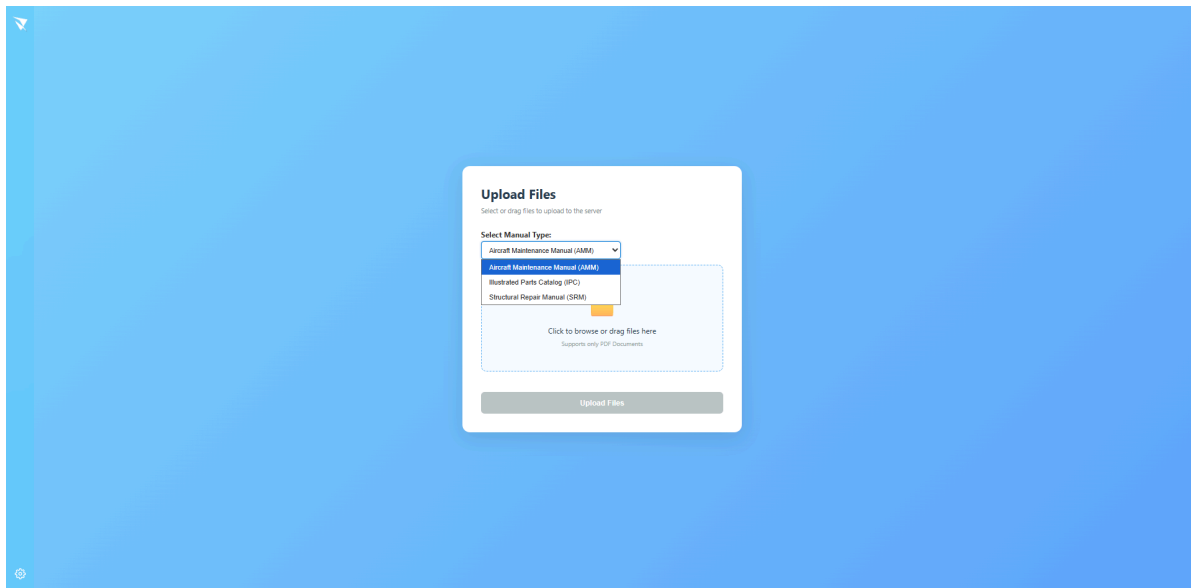


Fig. 6.7: File Uploading

Fig. 6.7 illustrates the file upload page, which is accessible through the left sidebar of the settings page (Audit Log). Access requires a password, which is hardcoded by the admins. Once authenticated, the user is directed to a page where manuals for AMM, IPC, or SRM can be uploaded.

During the upload process, the user must specify the manual type and ensure that all documents belonging to that type are uploaded, as the vector and blob databases are recreated for this feature. For example, when uploading AMM documents, every AMM document must be included. This approach prevents duplicate storage and embedding, while ensuring accurate and consistent retrieval of information.

2.3.4 Multiple Machine Accessibility

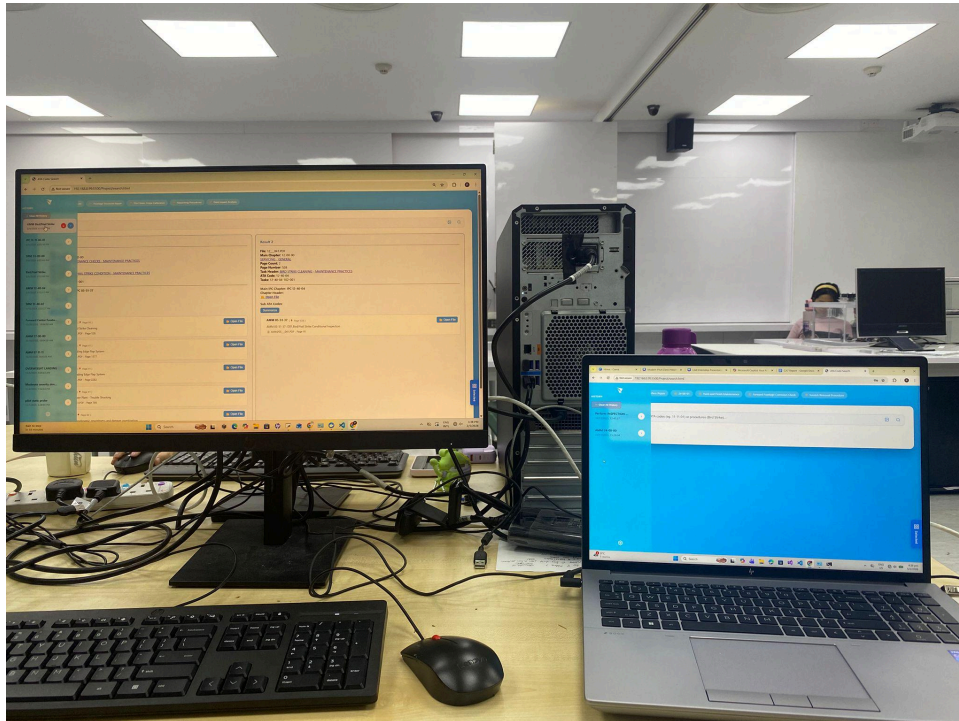


Fig. 7.1: A laptop accessing the web application by utilizing the computing power of the host computer

Fig. 7.1 illustrates the Multiple Machine Accessibility Feature, which enables several machines to connect to a single host computer while maintaining independent usage. Each connected machine can be accessed and operated individually, without interference from other connections. To ensure proper isolation and security, each machine must be accessed through a different account.

The host computer acts as a central hub, managing communication and resource allocation, while ensuring that each machine retains its own dedicated session. This design improves efficiency by reducing hardware overhead, enabling centralized control, and supporting scalable multi-user environments.

2.3.5 Other Additional Features

Streaming via FlaskAPI

```
yield f"data: {json.dumps({'new_code': code_entry, 'main_ata_code': main_ata_code})}\n\n"

progress = base_progress + (result_idx * progress_per_result) + \
    ((idx + 1) / total_pages) * progress_per_result
yield send_progress(int(progress), f"Extracting codes from page {page_num} ({idx+1}/{total_pages})")
```

Fig. 8.1: Streaming via FlaskAPI

FlaskAPI is used to facilitate communication between the frontend and backend. Fig. 8.1 illustrates how streaming is implemented via FlaskAPI. This mechanism enables real-time updates in the frontend, allowing the progress of an extraction or search to be continuously communicated to the user for better transparency. Furthermore, extracted ATA sub-codes are streamed and delivered as soon as they become available, enhancing the user experience by providing immediate feedback.

Temporary Saving of Sub ATA Code List

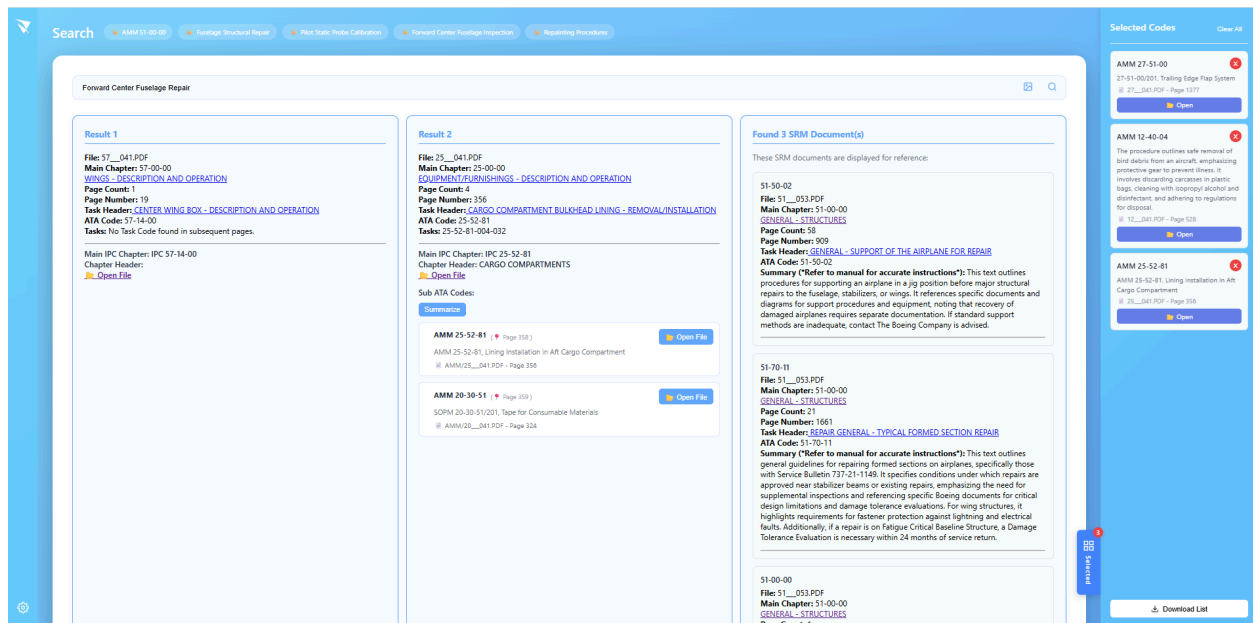


Fig. 8.2: Temporary Saving List

Fig. 8.2 shows the right sidebar that can be opened from the main page of the web application. Within this sidebar, users can drag and temporarily store extracted ATA sub-codes. The selected sub-codes can then be managed by removing them from the list, opening their associated files, or downloading a compiled list of the chosen sub-codes.

This feature provides a convenient way to organize and access specific sub-codes during the extraction process without losing track of important references.

Direct Search

For deeper exploration after an extraction is completed, users can click on the white space of any ATA sub-code box to initiate a direct search or trigger further extraction.

2.4 Operation

The web application was tested using a set of questions provided by engineers to evaluate the accuracy of the model. These questions included both AMM-specific and SRM-specific queries.

List of Questions:

AMM Queries

- Fuel Probe Test
- Capacitance Fuel Probe
- UHF Antenna
- Fire Extinguisher Bottles
- Lavatory Unserviceable
- Localiser Antenna

SRM Queries

- Crack on Windshield
- Radome Dent
- Inspection and Repair

Based on these test cases, the documents retrieved by the model were consistently accurate. However, recognizing that no system can achieve perfect accuracy, we conservatively estimate the model's accuracy to be 90% or higher.

Other features, such as the Damage Assessment via Image Search, were also tested by the clients to gain a deeper understanding of the system's capabilities. These tests evaluated the accuracy of image interpretation by the VLM and provided insights into the usability and overall UI/UX of the web application.

In terms of search time, the results are retrieved almost instantaneously. For extraction tasks, the estimated average time required is approximately 30 seconds, though this duration scales with the number of document pages being processed.

Current testing with two machines querying simultaneously showed no noticeable delay in response time. However, it is estimated that delays may begin to occur when four or more machines query the system concurrently.

The client reported no issues with the current web application during testing and has confirmed that it is ready for deployment. This indicates that the system has met the required functional and usability standards, and is prepared to transition from the testing phase into operational use.

3.0 Challenges Faced & Problem Solving

During my internship journey, I encountered several challenges.

- At the start of the internship, it was my first time working with RAG, and I had no prior experience or knowledge of the technology. As a result, I needed additional time to research and understand its concepts, which initially set back my progress. However, through continuous learning, experimentation, and guidance from supervisors, I gradually developed a solid understanding of RAG and was able to apply it effectively in my tasks.
- High latency from LLM calls was observed, as the system relies on semantic understanding with LLMs rather than simple keyword matching. Consequently, response times vary depending on the length and complexity of the manual section being processed. Longer chapters or multi-page queries may introduce noticeable delays compared to traditional keyword search engines. To mitigate this issue, streaming was implemented via FlaskAPI, allowing sub-ATA codes to be updated in real time as they are retrieved. This ensures that the frontend continuously reflects progress, providing users with immediate access to partial results and improving the overall experience.
- Another challenge lies in the LLM and hardware limitations. The web application currently runs on a locally hosted model (Ollama Phi4:14B), which contains 14 billion parameters and consumes a significant portion of computing resources. While this setup enables local control and avoids dependency on external APIs, it is inherently slower and less accurate compared to larger state-of-the-art models such as GPT-4, which is estimated to contain around 1.76 trillion parameters. This disparity in scale highlights the trade-off between accessibility and performance, as smaller models are more resource-efficient but cannot match the accuracy, speed, or generalization capabilities of larger models.
- During the deployment phase, I encountered difficulties with exporting and downloading the required Python libraries. This was particularly frustrating since the web application depends heavily on these dependencies to function correctly. To resolve the issue, I recreated the virtual environment and carefully reinstalled each library, ensuring compatibility and stability. This experience taught me the importance of being meticulous when managing dependencies and reinforced the need to verify environments thoroughly before deployment.

4.0 Outcome

The Aircraft Documentation Platform using Generative AI for Maintenance Manual Access has produced results across two domains:

4.1 Avotron Pte. Ltd.

Delivered a fully functional interactive web application tailored for aircraft maintenance engineers. The solution demonstrated significant improvements in retrieval accuracy by leveraging domain-specific chunking, clean preprocessing, and high-quality semantic embeddings, while also reducing search time compared to traditional methods. These results highlight the platform's potential to streamline maintenance workflows, enhance operational efficiency, and provide a scalable foundation for future enhancements.

4.2 5G&AIOT Centre

For the Centre, this project successfully showcased its capabilities in leveraging the latest technologies, particularly AIoT, for seamless integration. By demonstrating practical applications of Retrieval-Augmented Generation (RAG), Artificial Intelligence (AI) using Embedding Models, Large Language Models (LLMs) and Vision Language Models (VLMs) working together, the project not only validated the Centre's technological expertise but also established a reference framework. This foundation can guide future interns and research projects that aim to adopt similar advanced technologies.

4.3 Team Collaboration

Strengthened collaborative workflows with supervisors and fellow interns. Developed clear documentation, effective testing procedures, and iterative improvement strategies. Communication and presentation skills were enhanced through project updates and stakeholder demonstrations.

5.0 Impact

The Aircraft Documentation Platform using Generative AI for Maintenance Manual Access has impacted multiple stakeholders:

5.1 Avotron Pte. Ltd.

- By enabling engineers to instantly access the exact manual and relevant sections, organizations reduce document search time by up to **10 times (from ~5mins to ~30secs)**, allowing maintenance teams to dedicate more effort to actual troubleshooting and repair tasks.
- **Improved retrieval accuracy** by combining domain-specific chunking, clean pre-processing, and high-quality semantic embeddings.
- All manuals and sections are accessible in one place, having **consolidated knowledge**. The dashboard can grow with new manuals, being **scalable** and supporting long-term operational needs.
- Standardized retrieval ensures all specialists are working with the **same accurate information**. Validation is also required by specialists to ensure retrieved information is accurate.

Although not yet implemented, several enhancements have been identified. One potential improvement is the integration of AI-driven predictive maintenance, enabling the system to anticipate component failures or anomalies before they occur. This would reduce downtime, improve safety, and shift maintenance practices from reactive to proactive. Detected issues could then be processed by a LLM, which would generate relevant queries or troubleshooting steps through an AI Suggestion feature. Together, these capabilities would extend the platform beyond document retrieval, transforming it into a proactive decision-support tool for maintenance engineers.

5.2 5G&AIoT Centre

The project demonstrates the Centre's capability in integrating software and AI for real-world applications. It acts as a proof-of-concept for interactive user experiences and provides valuable learning opportunities for future interns, contributing to the Centre's research and development portfolio.

5.3 Other Fields/Industry Applications

Beyond Avotron Pte. Ltd., the project highlights the broader potential of combining retrieval systems for documentation with Generative Artificial Intelligence (Gen AI). It serves as a model for other industries, from marketing to maintenance, and even food & beverage by helping users instantly access relevant information, streamline workflows, and reduce time spent searching through large knowledge bases.

Overall, by consolidating domain knowledge and enabling semantic understanding, such systems can improve decision-making, enhance customer experiences, and support long-term operational efficiency across diverse sectors.

6.0 Key Skills

This section summarizes the key technical skills that readers may need to learn if they wish to develop a similar interactive system, covering programming, experimenting, prototyping, and web development.

6.1 Backend (Programming, Experimenting, Prototyping)

- **Python**
 - Integration with AI models, web application, databases
- **FlaskAPI**
 - Understanding endpoints
 - GET/POST requests
 - Streaming
- **Artificial Intelligence (AI)**
 - Computer Vision
 - Large Language Model (LLM)
 - Vision Language Model (VLM)
 - Prompt Engineering
- **Retrieval Augmented Generation (RAG) Logic**
 - Vectors, Qdrant (Vector Database)
 - Metadata
 - Cosine Similarity
 - Storing & Retrieving
 - Embedding Models
- **Deployment & Error Handling**
 - Docker
 - Logging/Debugging

6.2 Frontend (Web Development)

- **HTML**
 - Content
 - Structure
- **CSS**
 - Styling
- **JavaScript**
 - Scripting
- **Integration with Backend**
 - API calls
 - Handling JSON responses
- **DuckDB (Relational Database)**
- **MinIO (Blob Storage)**

7.0 Conclusion

During my internship at the 5G & AIoT Centre, I successfully developed an *Aircraft Documentation Platform using Generative AI for Maintenance Manual Access* for Avotron Pte. Ltd., leveraging Generative AI to improve access to maintenance manuals. The project combined advanced technologies such as RAG, LLMs, VLMs, embedding models, and computer vision, while integrating multiple databases including vector, blob, and relational into a unified web application. This solution allowed me to address real challenges faced in aviation maintenance documentation.

Through this experience, I acquired and improved valuable technical skills as mentioned in Section 6.0, such as,

- Python Programming (Backend, API)
- Artificial Intelligence (LLMs, VLMs, embedding models)
- Web Development (HTML, CSS, JS)

Beyond the technical outcomes, the experience strengthened my skills in backend programming, frontend development, and system integration, while also deepening my understanding of how to balance innovation with reliability. Working in a collaborative environment at the 5G & AIoT Centre further enriched the experience, exposing me to diverse perspectives and encouraging adaptability.

The project achieved its objectives by enabling engineers to instantly access the exact manual and relevant sections, organizations reduced document search time, allowing maintenance teams to dedicate more effort to actual troubleshooting and repair tasks. It demonstrated how combining software and AI technologies can generate innovative solutions that support the maintenance process. However, this technology of helping users instantly access relevant information can be used for other industries like marketing, sales and food & beverages.

For future development, the system could be enhanced by:

- Predictive Maintenance - Using AI to generate solutions for predicted defects on aircrafts
- Faster search time
- Improved AI model accuracy
- Enhance user experience
- Integrating Parts Catalog
- Pre-Scheduling

Overall, this internship has been a highly rewarding experience, equipping me with practical skills and insights that will serve as a strong foundation for my professional career.

8.0 Additional Project - Smart AI Advertiser

During my internship at the 5G & AIoT Centre, I also had the opportunity to contribute to an additional project alongside fellow interns: the development of a Smart AI Advertiser for Vanskee Enterprise (S) Pte Ltd, a Singapore-based company established in 1990 that specializes in RFID and NFC solutions.

Drawing on the knowledge and experience gained from my main project, I was able to apply technologies such as Retrieval-Augmented Generation and Generative AI to this advertising solution while learning new ones.

Below are some information of the additional project and my contribution in the backend:

8.1 Scope

The goal was to create an AI-enabled exhibition booth that engages visitors by providing personalized beer recommendations based on their preferences.

Key deliverables:

- An interactive agent utilizing AI models for user interaction,
- A recommendation engine for tailored beer suggestions, and
- Fully integrated booth hardware to support real-time interaction and immersive visitor experiences.

8.2 Purpose

The system aims to attract visitors, increase brand visibility, and promote products by delivering personalized beer recommendations. By consolidating multiple functions into a single, effective platform, it also helps reduce manpower requirements. Overall, the solution is designed to enhance customer engagement, strengthen brand awareness, and support key marketing goals.

8.3 Problem Statement

Emerging brands like AsaBrew often face difficulties in standing out at exhibitions, where many competitors compete for visitor attention. Traditional booth setups typically rely on static displays or simple product sampling, which may not fully capture interest or motivate visitors to try unfamiliar products. In addition, booth staff face challenges in simultaneously promoting the brand, recommending products, and handling operations efficiently. This creates a gap for an engaging, all-in-one solution that not only attracts visitors but also delivers personalized recommendations, streamlines booth activities, and strengthens overall brand visibility.

8.4 Sketch & Solution

Solution

A smart interactive agent acts like a virtual avatar that provides personalised user experience with advertisement.

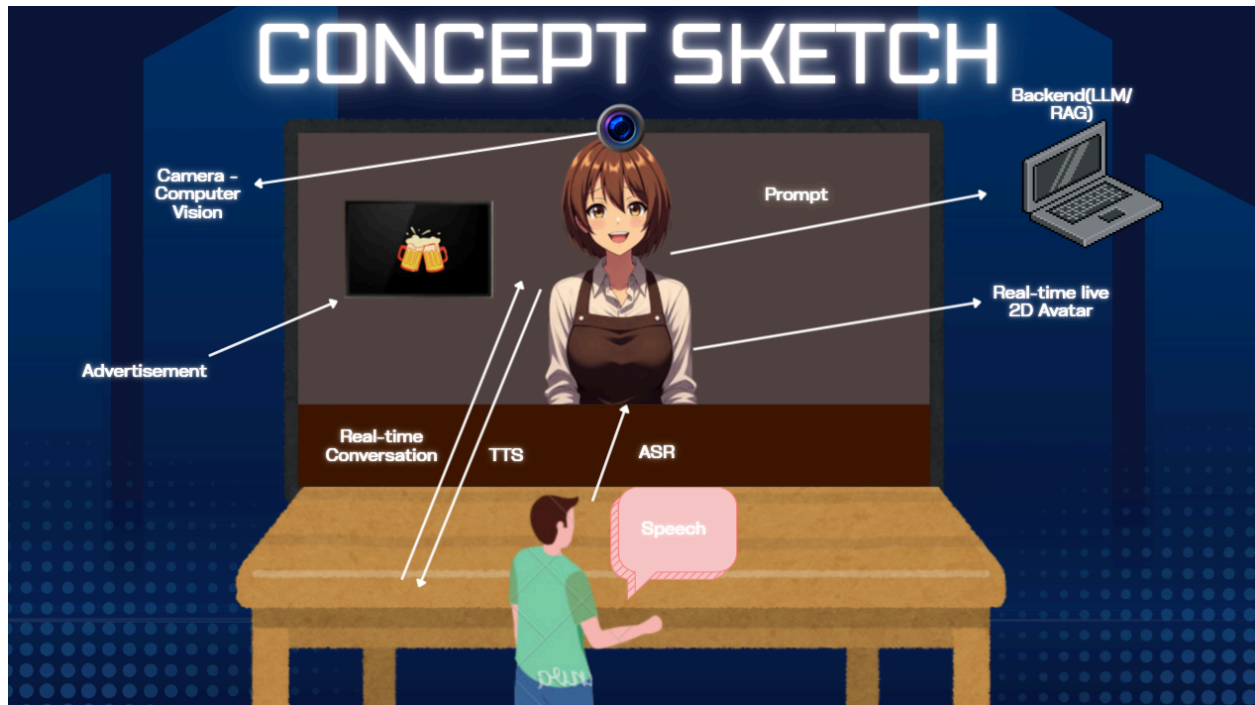


Fig. 9.1: Concept Sketch

This concept sketch provides an early visualization of how the booth setup and system components are integrated. It illustrates the arrangement of the display screen and camera for computer vision, while highlighting the use of Automatic Speech Recognition (ASR) and Text-to-Speech (TTS) to enable real-time communication between the AI-generated 2D avatar and the user. A backend powered by a Large Language Model (LLM) and Retrieval-Augmented Generation (RAG) supports personalized recommendations and chatbot functionality, ensuring that the avatar can respond intelligently and contextually during live interactions.

8.5 Data Flow

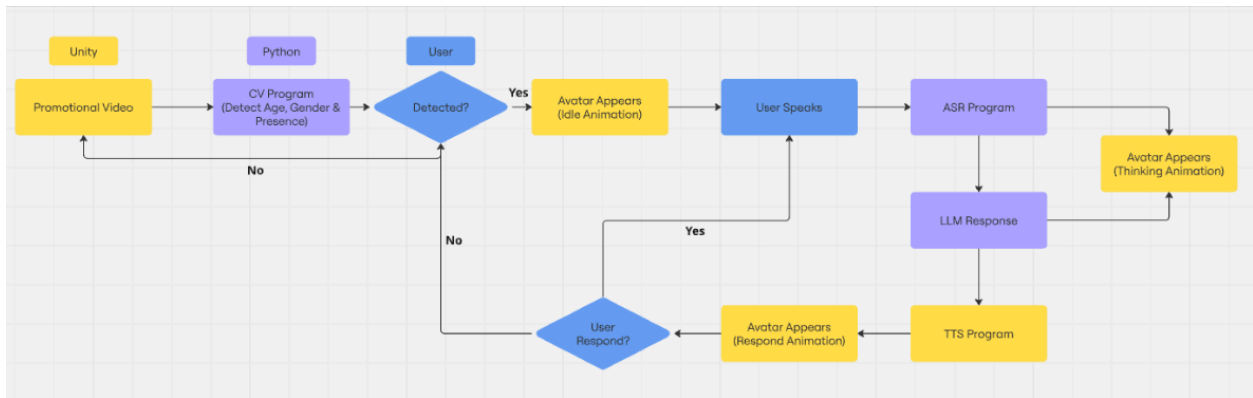


Fig. 9.2: Data Flow

8.6 System Architecture

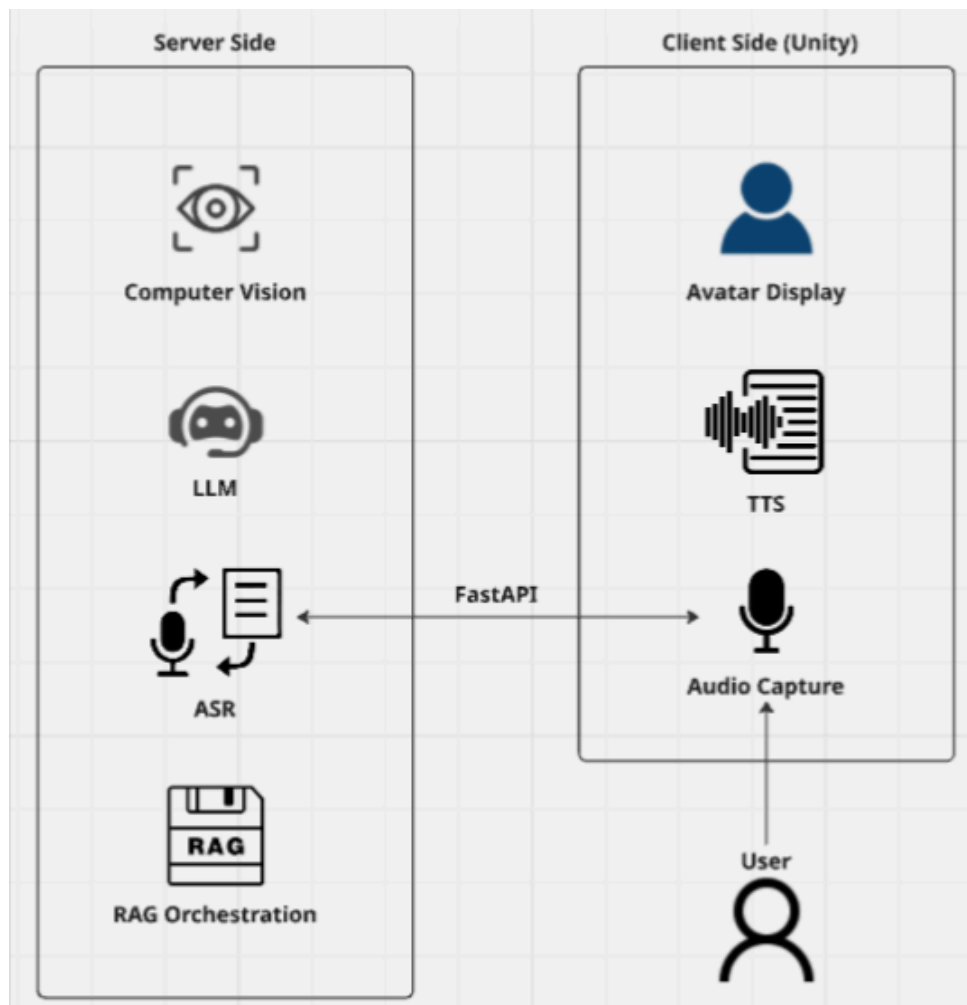


Fig. 9.3: System Architecture

8.7 My Contribution

I contributed primarily to the backend logic of the Smart AI Advertiser project, which aligned closely with the skills I had developed in my main internship project. Since the system relied on Retrieval-Augmented Generation (RAG) and generative AI technologies, it was a natural extension of my earlier work. This gave me valuable practice in implementing these methods in a new context, where instead of aviation manuals, the focus was on delivering personalized advertising content through a real-time avatar interface. My role involved ensuring that user prompts were processed effectively, linked to relevant contextual information, and returned as engaging responses that could support both chatbot interactions and targeted recommendations.

8.8 Contribution Features

Features:

- **Conversational Chatbot**
 - LLM (Large Language Model), ASR (Automatic Speech Recognition), and TTS (Text-to-Speech) work together to create human-like chatbot interactions that feel natural and engaging. ASR allows the system to capture spoken input accurately, while the LLM processes the text to generate intelligent, context-aware responses. TTS then converts these responses back into speech, enabling real-time two-way communication.
- **Computer Vision**
 - Computer vision enhances the system by detecting key user attributes such as presence, age, and gender. This information allows the platform to dynamically assign avatars that feel more relatable to each individual, creating a stronger sense of connection during interactions. By tailoring the avatar's appearance and responses to the user profile, the system can deliver more personalized recommendations whether in advertising, customer service, or training contexts.
- **Recommendations**
 - Retrieval-Augmented Generation (RAG) ensures that information delivery is not only precise but also engaging and contextually relevant. By combining semantic search with generative AI, RAG retrieves the most relevant sections of documentation or knowledge bases and enriches them with natural language responses. This allows the system to provide users with accurate, concise, and easy-to-understand information rather than overwhelming them with raw random data.

8.9 Pictures

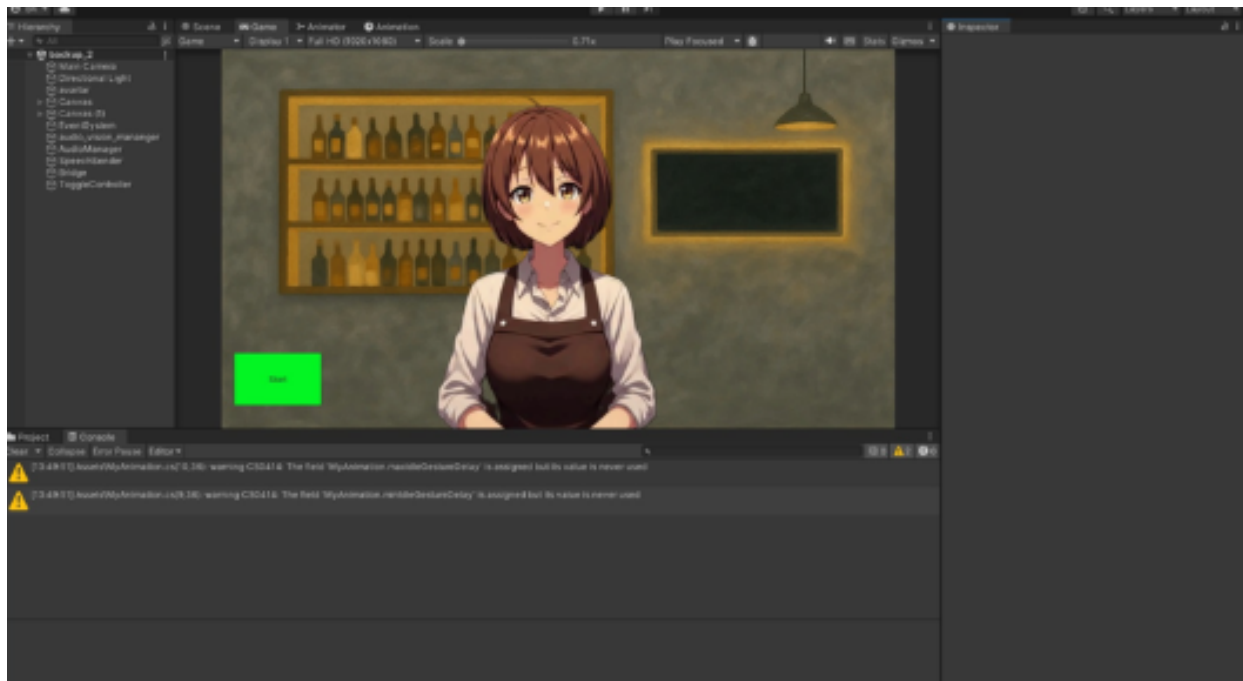


Fig. 9.4: Unity 2D Avatar

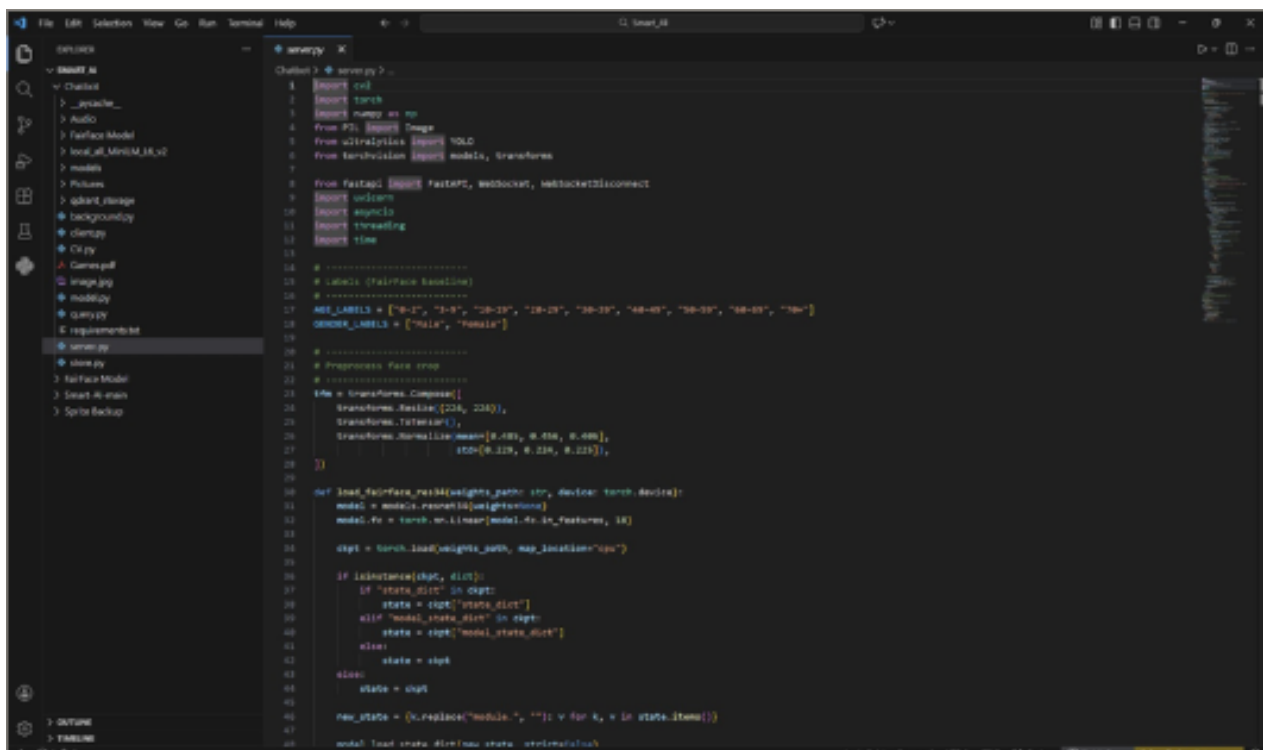


Fig. 9.5: Backend Logic for Computer Vision

9.0 References & Learning Resources

9.1 Learning Resources & Documentation

- [Python Official Docs](#)
- [Ollama](#)
- [HTML, CSS, JavaScript \(Web Development\)](#)
- [Qdrant \(Vector Database\)](#)
- [DuckDB \(Relational Database\)](#)
- [MinIO \(Blob Storage\)](#)
- [Computer Vision \(Roboflow\)](#)

9.2 Backend Development

- [Retrieval Augmented Generation \(RAG\) Pipeline](#)
- **Integration with Python:**
 - [Qdrant](#)
 - [DuckDB](#)
 - [MinIO](#)
 - [Ollama](#)
 - [Computer Vision \(Roboflow\)](#)

9.3 AI Models

- [Large Language Model - Ollama phi4](#)
- [Vision Language Model - Ollama qwen2.5vl](#)
- [Embedding Model - HuggingFace all-MiniLM-L6-v2](#)
- [Computer Vision for Aircraft Defects - Roboflow \(Open Source\)](#)

9.4 Development Tools & Installation Guide

- [Visual Studio Code](#)
- [Docker](#)
- [Ollama](#)
- [Python](#)
- [Qdrant](#)
- [MinIO](#)
- [DuckDB](#)

9.5 Additional Tools

- [Miro \(Flowcharts, Diagrams\)](#)
- [Canva \(Presentation, Slides\)](#)
- [Figma \(Web Development Design/Sketch\)](#)

10.0 Appendix

10.1 Deployment Instructions

Download and install the following tools, **Ollama** (on CUDA), **Docker**, **Python**, **Visual Code Studio**. (Refer to Section 8.4 for Instruction Guide)

Note: This web application is suited for Boeing 737-300/400/500 AMM, SRM and IPC.

Once each software is downloaded (*Note: The commands provided are intended for PCs with a D: drive.*):

10.1.1 Ollama

- Open up command prompt on the PC and run these commands:
 - `ollama run phi4:14b`
 - `ollama run qwen2.5vl:7b`

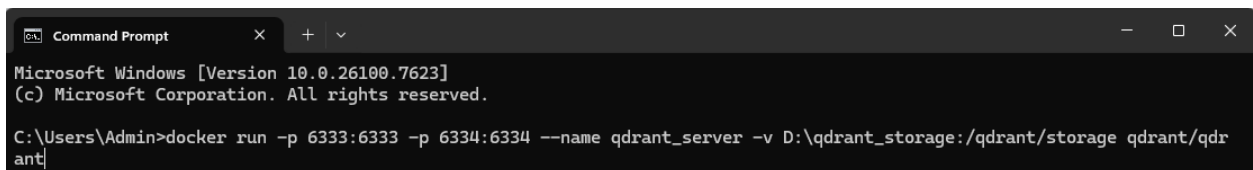


```
Microsoft Windows [Version 10.0.26100.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin>ollama run phi4:14b
```

10.1.2 Docker

- Open up command prompt on the PC and run these commands:
 - `docker run -p 6333:6333 -p 6334:6334 --name qdrant_server -v D:\qdrant_storage:/qdrant/storage qdrant/qdrant`
 - `docker run -p 9000:9000 -p 9090:9090 --name minio_server -v D:\miniodata:/data -e "MINIO_ROOT_USER=minioadmin" -e "MINIO_ROOT_PASSWORD=minioadmin" minio/minio:latest server /data --console-address ":9090"`



```
Microsoft Windows [Version 10.0.26100.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin>docker run -p 6333:6333 -p 6334:6334 --name qdrant_server -v D:\qdrant_storage:/qdrant/storage qdrant/qdrant
C:\Users\Admin>docker run -p 9000:9000 -p 9090:9090 --name minio_server -v D:\miniodata:/data -e "MINIO_ROOT_USER=minioadmin" -e "MINIO_ROOT_PASSWORD=minioadmin" minio/minio:latest server /data --console-address ":9090"
```

After installing:

1. Ensure that the Docker containers are running.
2. In Docker, click on the underlined text labeled “**9090:9090**” or “**9000:9000**” to open MinIO.
3. Log in using “minioadmin” as both the username and the password.
4. Create a new bucket named “**aircraft**.”
5. Inside the bucket, set up folders for each manual type (e.g., AMM, IPC, SRM). Upload the corresponding documents into their respective folders.

Create Bucket

Filter Buckets

Buckets

aircraft

aircraft2

aircraft

Created on: Mon, Dec 01 2025 09:14:15 (GMT+8) Access: CUSTOM 584.3 MiB - 352 Objects

<

aircraft

☐

▲

Name

☐

AMM

☐

IPC

☐

SRM

aircraft

Created on: Mon, Dec 01 2025 09:14:15 (GMT+8) Access: CUSTOM 584.3 MiB - 352 Objects

<

aircraft / AMM

☐	Name	Last Modified
☐	 05__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 06__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 07__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 08__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 09__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 10__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 11__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 12__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 20__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 21__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 22__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 23__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 24__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 25__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)
☐	 26__041.PDF	Wed, Jan 14 2026 15:10 (GMT+8)

10.1.3 Python

- Open up command prompt and run this command:
 - `py install 3.12`

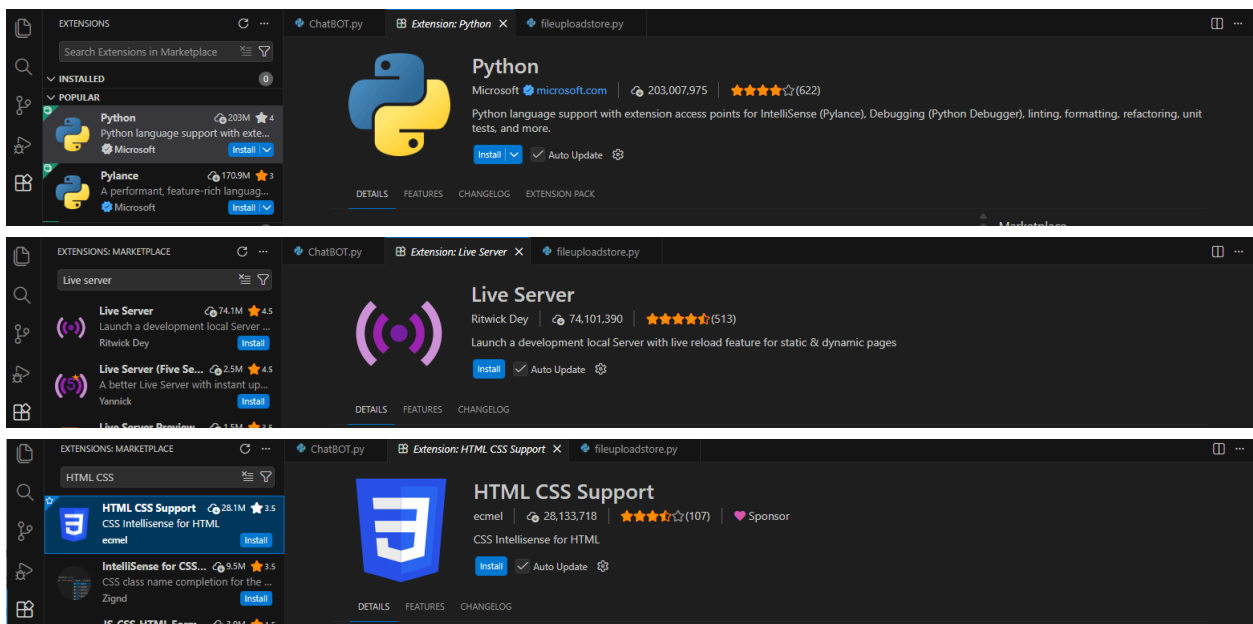


```
Microsoft Windows [Version 10.0.26100.7623]
(c) Microsoft Corporation. All rights reserved.

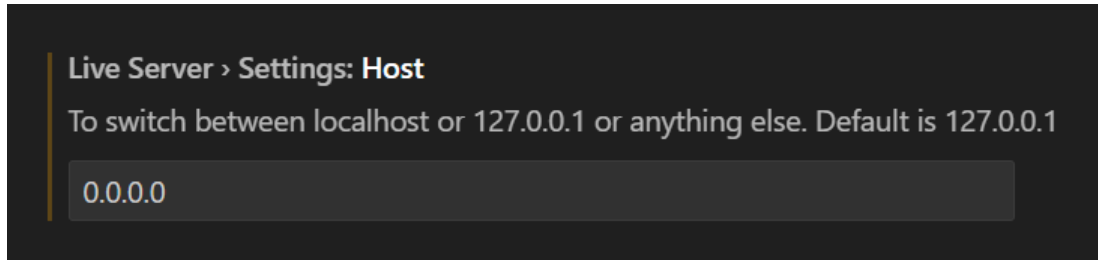
C:\Users\Admin>py install 3.12
```

10.1.4 Visual Studio Code

- Open up the software, go to extensions on the left hand side and download these extensions:
 - Python
 - Live Server (by Ritwick Dey)
 - HTML CSS Support
 - Code Runner (Optional, Easier to run code)



- Open Visual Studio Code Settings located at the bottom left.
- Search for “**live server host.**”
 - The default value should appear as **127.0.0.1**.
 - Change this value to **0.0.0.0**.



- Next, search for “**live server ignore files.**”
- Edit the **settings.json** file to include “***”, under “**liveServer.settings.ignoreFiles**”:

```
{  
  "liveServer.settings.CustomBrowser": "chrome",  
  "liveServer.settings.host": "0.0.0.0",  
  "liveServer.settings.ignoreFiles": [  
    "***",  
    ".vscode/**",  
    "**/*.scss",  
    "**/*.sass",  
    "**/*.ts"  
  ]  
}
```

- Unzipped the zipped “**Internship-main**” file and move it to D: drive so the path would look like this “**D:\Internship-main**”, inside the folder should contain a Project folder which contains all the logic and web application.
- After, open up the project folder, “**Internship-main**”, under “**File**” and “**Open Folder**”.
- Open a new terminal in Visual Studio Code under “**Terminal**” and “**New Terminal**”.
- Type the following commands:
 - `setx PYTHONUTF8 1`
 - `python --version` (Ensure it's Python Version 3.12.10)
 - `python -m pip install -r requirements.txt`

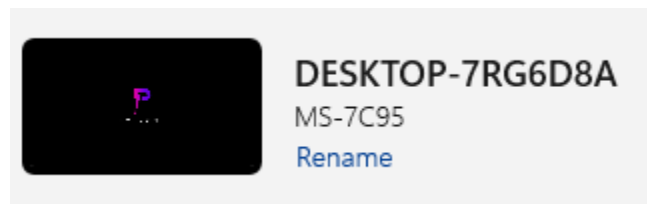
```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Admin\Downloads\Internship-main\Internship-main> setx PYTHONUTF8 1

```

- Under the folder on the left hand side, the files are shown.
- In the files, “**ChatBOT.py**” & “**fileuploadstore.py**”, the name after “**DESKTOP =**” needs to be changed to the host machine name. (You can find the name by opening settings on the PC/Laptop)



```
DESKTOP = "DESKTOP-7RG6D8A"
```

- Repeat the previous step with all “.html” files for “**const DESKTOP =**”

```
const DESKTOP = "DESKTOP-7RG6D8A";
```

10.1.5 Launching Web Application

- Before launching the web application, run “**Qdrant.py**” ONCE on the host machine by typing in Visual Studio Code terminal or click the top right play button if you installed “Code Runner”:

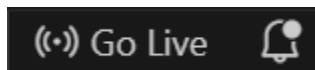
- `python Project/Qdrant.py`

This step initializes the vector database required for storing and retrieving embeddings.

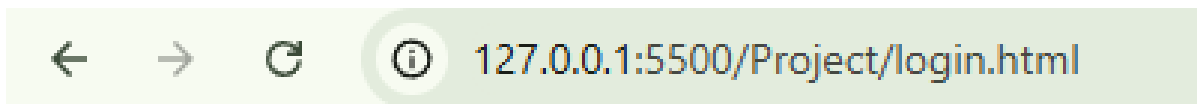
```
PS D:\Internship-main> python Project/Qdrant.py
```

1. Run both “**ChatBOT.py**” & “**fileuploadstorage.py**” using “`python Project/(filename)`” in the separate terminal or command prompt.

Or optionally, open start_app.bat in file explorer to run the files skipping step 2 - 4. And Click the “Go Live” icon at the bottom right.



2. Open “**login.html**” in Visual Studio Code.
3. Right-click the code and select “**Open with Live Server.**”
4. A browser window will open. If the page displays “The site can’t be reached,” update the URL:
 - Replace **0.0.0.0** with **127.0.0.1** when running on the host machine.
 - Alternatively, use the host machine’s **IP address** if you want to access the application across multiple devices.
5. Upload files and test web application features.



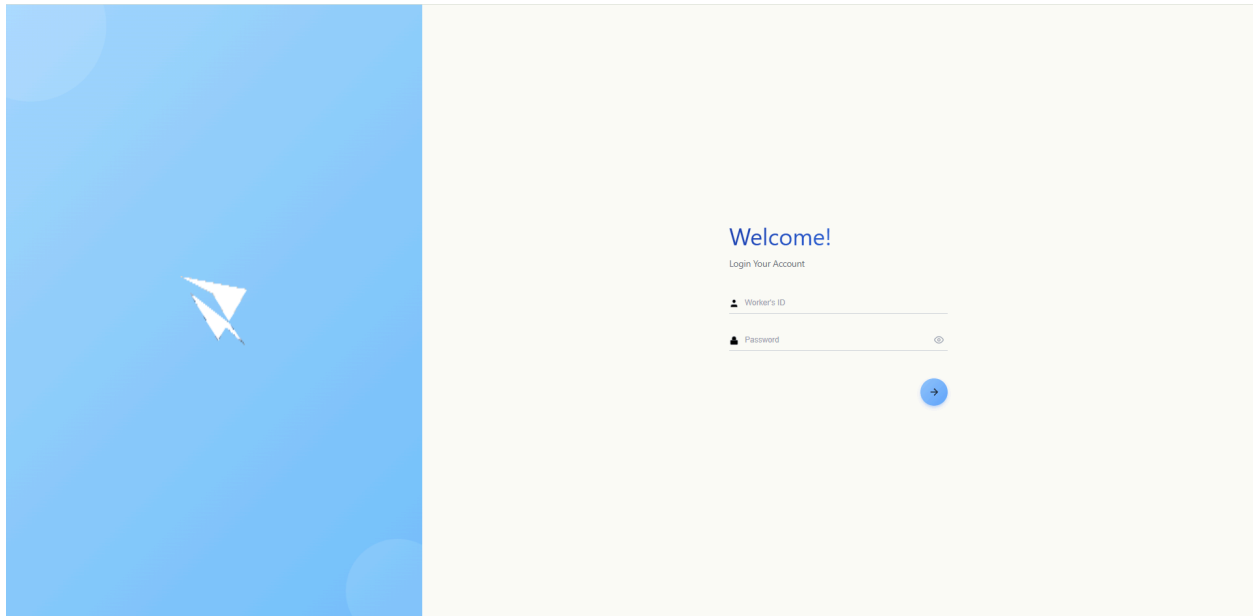
10.2 Web Application Instructions

This section shows the instructions for the features in the web application.

- At any point of time, click the logo at the top left of the left sidebar to go back to the main page.

10.2.1 Login

Username & Password are set inside the login.html code.

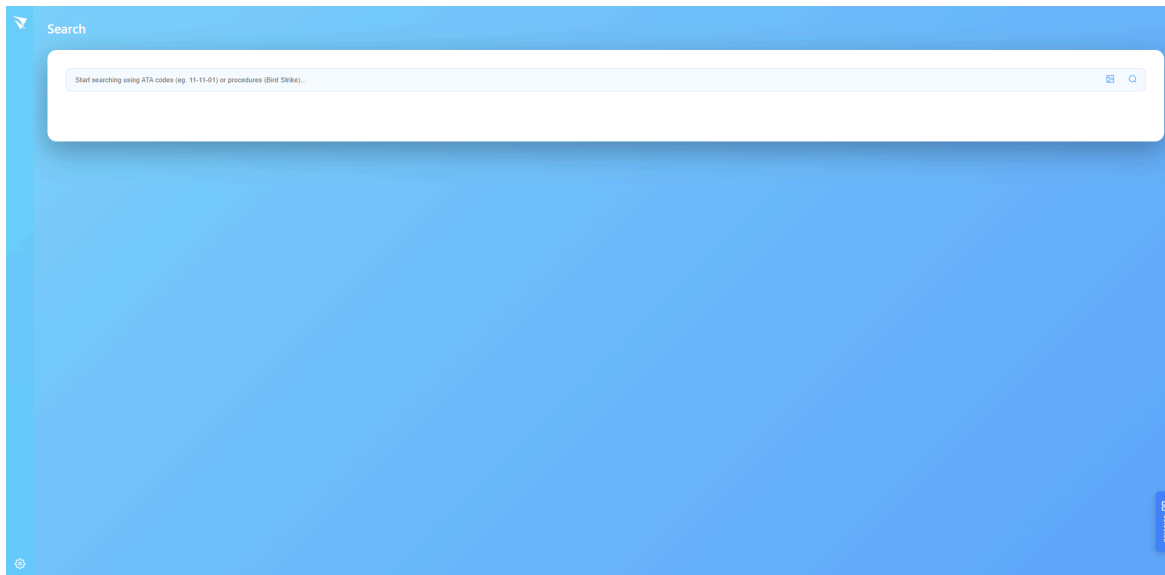


10.2.2 Search

The web application allows users to type natural language queries into the search bar to ask questions about the manuals or maintenance procedures. Above the search bar, AI-generated suggestions based on the user's search history are displayed, enabling quick access to commonly searched topics. After performing a search, the user can validate results by accessing the corresponding files, headers, and pages of the ATA code manual and main chapters.

Examples of queries include:

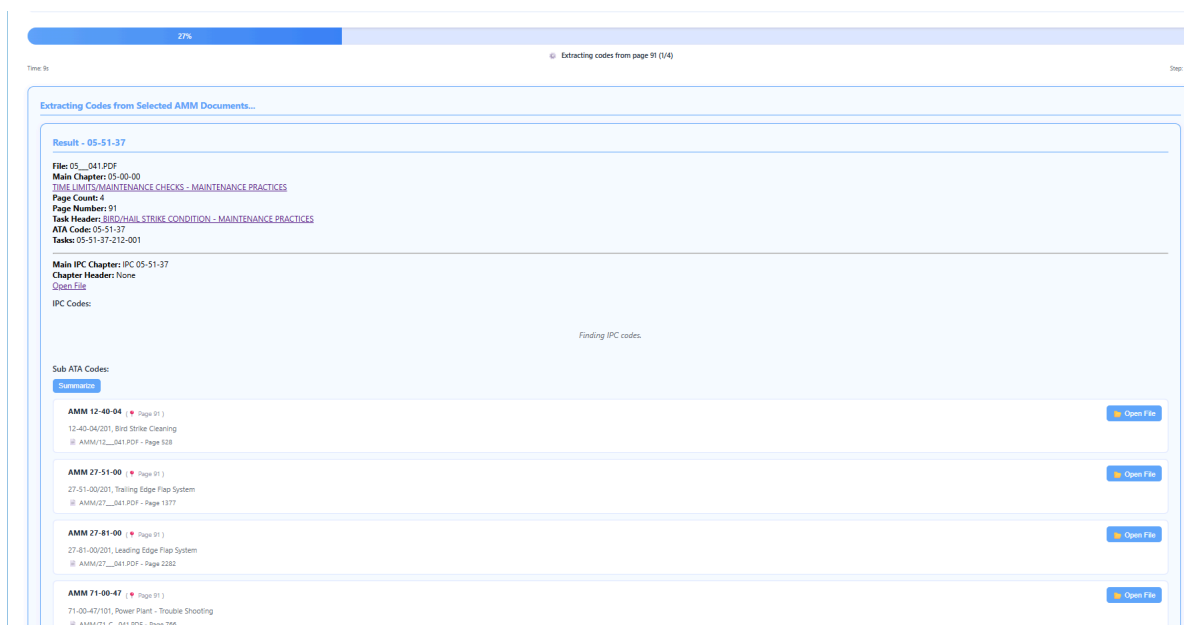
- "How to fix bird strike?" -> AMM/SRM
- "AMM How to fix bird strike?" -> AMM Only (Instant Search for Validation)
- "AMM XX-XX-XX" -> AMM Direct Chapter (Instant Search for Extraction)
- "IPC XX-XX-XX-XXX" -> IPC Direct Chapter (Instant Search)
- "SRM XX-XX-XX" -> SRM Direct Chapter



10.2.3 Sub ATA Code Retrieval/Extraction

Users can choose specific ATA codes within the AMM to extract their corresponding sub-ATA codes. After making a selection, clicking the **“Generate Sub-Codes for Selected”** button initiates the retrieval and extraction process.

- Sub-Codes Summarization: Users can click on the "Summarize" button to generate a rough summary of each sub-code manual page.
- Direct Search by Clicking: Users can click on the white spaces of each sub-code to directly search and extract the sub-code selected.



10.2.4 History

Users can hover over the left side of the page to open the history panel, which stores all previous searches. Each entry displays the search query along with the date and time it was performed. Clicking on an entry reopens that search, while clicking the cross (X) button removes it from the history. Users can also clear the entire history at once by selecting the “Clear All History” button.

The screenshot displays a software interface with a blue header bar containing navigation tabs: "Roadmap - Structural Repair", "Pilot Static Probe Calibration", "Overweight Landing Procedures", and "Surface Smoothness Verification". Below the header, the "HISTORY" panel is open on the left, showing a list of search entries with their respective dates and times. A "Clear All History" button is at the top of this panel. The main content area on the right shows the results of a selected search, displaying three SRM documents with their file names, main chapters, page counts, and task headers. The documents are titled "57-00-00", "51-20-06", and "57-00-03". Each document entry includes a summary of its content, such as "Summary (Refer to manual for accurate instructions): The text outlines procedures and regulations related to maintaining aircraft wing integrity against bird/hail strikes." The interface also features a "Generate Sub-Codes for Selected" button and "Select All" and "Deselect All" buttons at the bottom of the main content area.

HISTORY:

- Clear All History**
- AMM Bird/Hail Strike**
2/1/2025, 6:45:00 PM
- IPC 11-11-00-01**
2/4/2025, 6:03:16 PM
- SRM 53-00-00**
2/4/2025, 6:02:48 PM
- Bird/Hail Strike**
2/4/2025, 5:12:58 PM
- AMM 12-40-04**
2/4/2025, 5:12:11 PM
- SRM 51-40-02**
1/13/2025, 2:02:27 PM
- Forward Center Fusel...**
12/30/2025, 10:04:59 AM
- AMM 27-00-00**
1/10/2025, 10:04:29 AM
- AMM 07-11-11**
12/30/2025, 9:43:03 AM
- OVERWEIGHT LANDING**
12/4/2025, 2:29:40 AM
- Moderate severity den...**
12/4/2025, 1:54:34 PM
- pilot static probe**
12/17/2025, 2:29:08 PM
- 51-10-00**
11/28/2025, 2:13:08 PM

Found 3 SRM Document(s)

These SRM documents are displayed for reference:

57-00-00
File: 57_..._053.PDF
Main Chapter: 57-00-00
GENERAL - WINGS
Page Count: 4
Page Number: 30
Task Header: GENERAL - WINGS
ATA Code: 57-00-00
Summary (Refer to manual for accurate instructions): The text outlines procedures and regulations related to maintaining aircraft wing integrity against bird/hail strikes. It emphasizes compliance with CDCLs to prevent fuel tank ignition, detailing fastener requirements for lightning protection in specific zones. Adherence to manufacturer's maintenance procedures is crucial to avoid risks of fire or explosion, ensuring safety as per FAA guidelines.

51-20-06
File: 51_..._053.PDF
Main Chapter: 51-00-00
GENERAL - STRUCTURES
Page Count: 2
Page Number: 270
Task Header: GENERAL - SHOT PEEING
ATA Code: 51-20-06
Summary (Refer to manual for accurate instructions): For bird or hail strikes, shot peening should be applied to aluminum, steel, and titanium parts after removing damage. Ensure the part's thickness is above minimum values in Table 1/GENERAL before peening. Adjust intensity based on remaining material thickness post-damage removal. Avoid peening if below these thresholds. Refer to SCIPM 20-10-03 for detailed data.

57-00-03
File: 57_..._053.PDF
Main Chapter: 57-00-00
GENERAL - WINGS
Page Count: 30
Page Number: 34
Task Header: ALLOWABLE DAMAGE 1 - WING STRINGERS
ATA Code: 57-00-03
Summary (Refer to manual for accurate instructions): The text outlines permissible damage limits for center and outer wing stringers due to bird or hail strikes as detailed in Figure 101 under

Generate Sub-Codes for Selected **Select All** **Deselect All**

10.2.5 Defect Assessment via Image Search and Location Selection

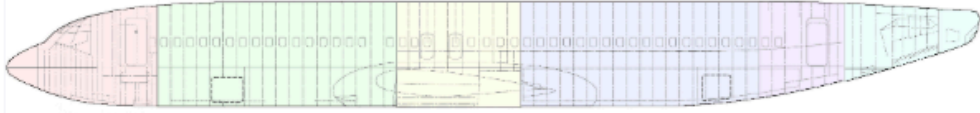
Within the search bar, users have the option to click on an image icon that opens a dedicated page for defect assessment. This page allows them to specify the location of the defect by interacting with a colored aircraft diagram, which can be enlarged for easier selection. The enlarged image can be closed using the cross button at the top right.

Alternatively, users may choose to indicate the defect location by entering the station and section details, or they can leave these fields blank if they prefer to proceed with only a damage assessment.

Below the location selection, users are provided with a box to upload an image of the defect along with a text field for additional comments.

Once the necessary inputs are completed, clicking the **“Search”** button initiates the assessment process. After a short while, the system generates a prompt summarizing the findings, which may include the type of defect, its location (if specified), the severity of the damage, and any comments provided. Based on this information, the relevant AMM and SRM manuals are retrieved to support further analysis and validation.

Upload Image



Click image to enlarge and mark defects

Station:

Section:

-- Select Section --



Click to upload an image

PNG, JPG, JPEG up to 10MB

Additional Instructions (Optional)

Add any specific details or questions about the image...

Cancel

Search

10.2.6 Temporary Saving of Selected ATA Sub-Codes

At the bottom right of the page, users will find a **“Selected”** button that opens a sidebar when clicked. This sidebar serves as a temporary workspace where users can drag and store the ATA sub-codes they wish to keep for further reference. Within this panel, users have the flexibility to manage their selections:

- Remove sub-codes from the list,
- Open the associated files directly
- Download a compiled list of all the sub-codes they have chosen

The screenshot displays a web application interface for searching ATA sub-codes. The top navigation bar includes a search bar and several filter tabs: "Airfield 51-00-00", "Fuselage Structural Repair", "Main Cabin Module Calibration", "Forward Center Fuselage Inspection", and "Repairing Procedures".

The main content area is divided into three columns:

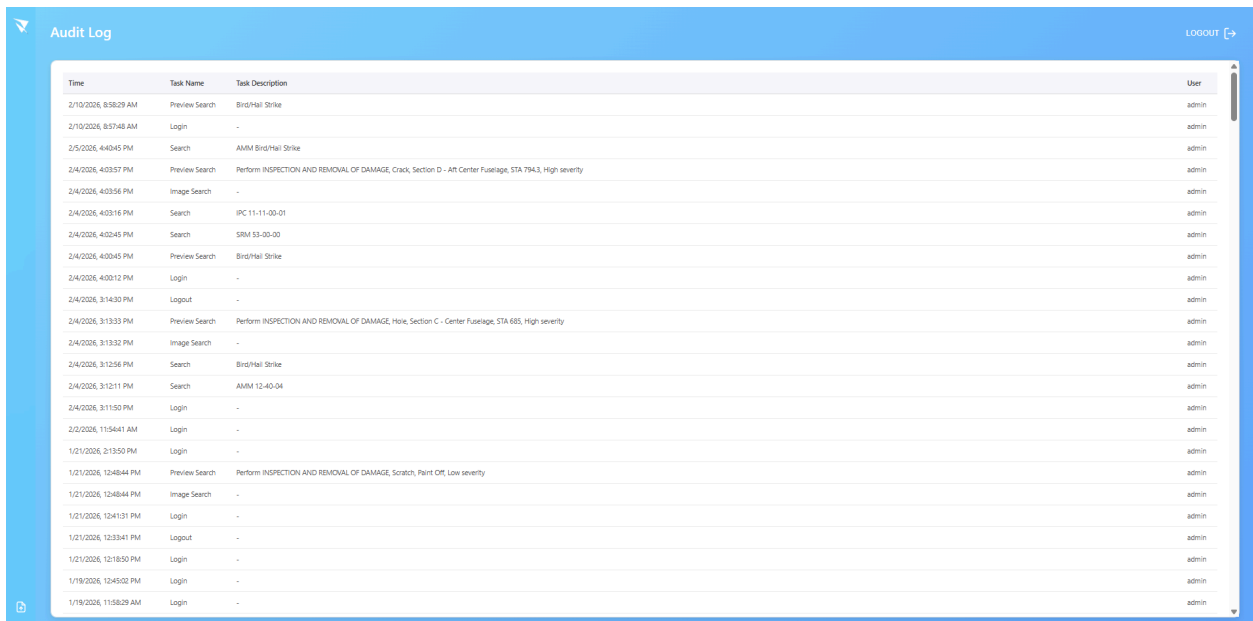
- Result 1:** Displays details for File 57_041.PDF, including Main Chapter (57-00-00), WINGS - DESCRIPTION AND OPERATION, Page Count (1), Page Number (19), Task Header (CENTER WING BOX - DESCRIPTION AND OPERATION), ATA Code (57-14-00), and a note that no task code was found in subsequent pages. It also shows the Main IPC Chapter (IPC 57-14-00), Chapter Header, and an "Open File" button.
- Result 2:** Displays details for File 25_041.PDF, including Main Chapter (25-00-00), EQUIPMENT/FURNISHINGS - DESCRIPTION AND OPERATION, Page Count (4), Page Number (356), Task Header (CARGO COMPARTMENT BULKHEAD LINING - REMOVAL/INSTALLATION), ATA Code (25-52-81), and Task (25-52-81-004-032). It also shows the Main IPC Chapter (IPC 25-52-81), Chapter Header (CARGO COMPARTMENTS), Sub ATA Codes, and "Summary" and "Open File" buttons.
- Found 3 SRM Document(s):** A section titled "These SRM documents are displayed for reference:" containing three document entries with their respective ATA codes and summaries.

On the right side, a sidebar titled "Selected Codes" lists three selected sub-codes: AMM 27-51-00, AMM 12-40-94, and AMM 25-52-81. Each entry includes a brief description and an "Open" button. At the bottom of the sidebar, there is a "Download List" button.

10.2.7 Audit Log

By hovering over the left side of the page, users can open the sidebar that contains both the search history and additional tools. At the bottom left of this sidebar, a settings icon is available. Clicking on this icon directs the user to the audit log, which provides a detailed record of activity within the system. Currently, it tracks:

- Login
- Logout
- Preview Search (Validation)
- Image Search (Damage Assessment)
- Full Search (Extraction)



The screenshot shows the 'Audit Log' interface. It features a blue header bar with a 'Logout' button on the right. Below the header is a table with four columns: 'Time', 'Task Name', 'Task Description', and 'User'. The table contains 20 rows of activity logs, including login, logout, and various search tasks (Preview Search, Image Search, Full Search) performed by the 'admin' user. The tasks are categorized by type (Login, Logout, Preview Search, Image Search, Search) and include specific timestamps and descriptions of the actions taken.

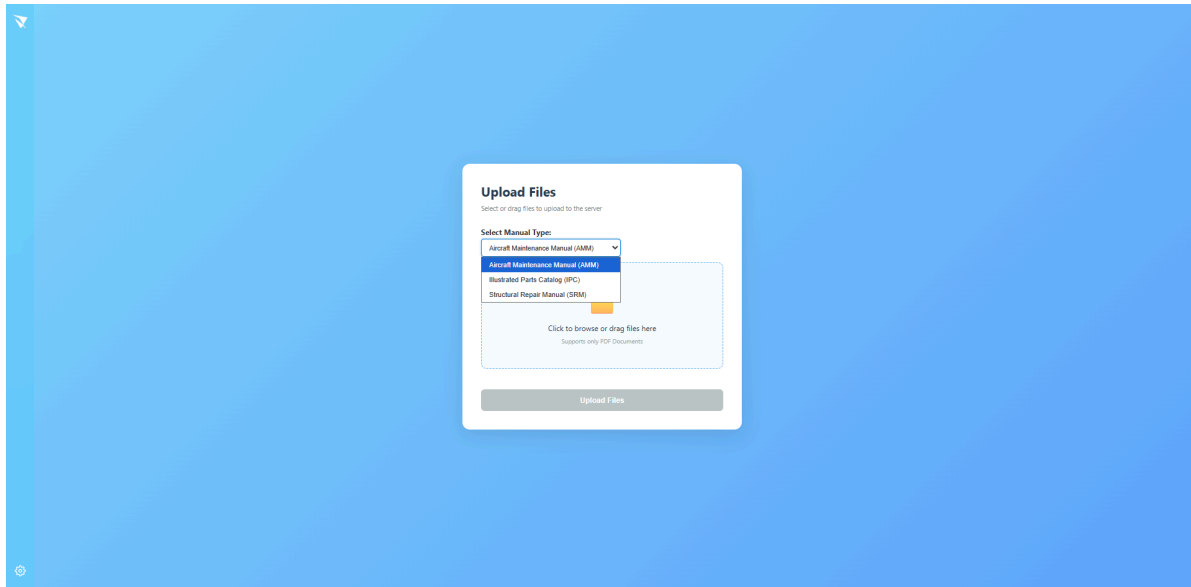
Time	Task Name	Task Description	User
2/10/2020, 9:58:29 AM	Preview Search	Bird/Hail Strike	admin
2/10/2020, 9:57:48 AM	Login	-	admin
2/5/2020, 4:40:45 PM	Search	AMM Bird/Hail Strike	admin
2/4/2020, 4:03:57 PM	Preview Search	Perform INSPECTION AND REMOVAL OF DAMAGE, Crack, Section D - Aft Center Fuselage, STA 794.3, High severity	admin
2/4/2020, 4:03:56 PM	Image Search	-	admin
2/4/2020, 4:03:16 PM	Search	IPC 11-11-00-01	admin
2/4/2020, 4:02:45 PM	Search	SRM 53-00-00	admin
2/4/2020, 4:02:45 PM	Preview Search	Bird/Hail Strike	admin
2/4/2020, 4:02:12 PM	Login	-	admin
2/4/2020, 3:14:30 PM	Logout	-	admin
2/4/2020, 3:13:33 PM	Preview Search	Perform INSPECTION AND REMOVAL OF DAMAGE, Hole, Section C - Center Fuselage, STA 685, High severity	admin
2/4/2020, 3:13:32 PM	Image Search	-	admin
2/4/2020, 3:12:56 PM	Search	Bird/Hail Strike	admin
2/4/2020, 3:12:11 PM	Search	AMM 12-40-04	admin
2/4/2020, 3:11:50 PM	Login	-	admin
2/2/2020, 11:54:41 AM	Login	-	admin
1/21/2020, 2:13:50 PM	Login	-	admin
1/21/2020, 12:48:44 PM	Preview Search	Perform INSPECTION AND REMOVAL OF DAMAGE, Scratch, Paint Off, Low severity	admin
1/21/2020, 12:48:44 PM	Image Search	-	admin
1/21/2020, 12:41:31 PM	Login	-	admin
1/21/2020, 12:33:41 PM	Logout	-	admin
1/21/2020, 12:18:50 PM	Login	-	admin
1/19/2020, 12:45:02 PM	Login	-	admin
1/19/2020, 11:58:29 AM	Login	-	admin

10.2.8 Logout

On the top right of the settings page, there is a “Logout” button.

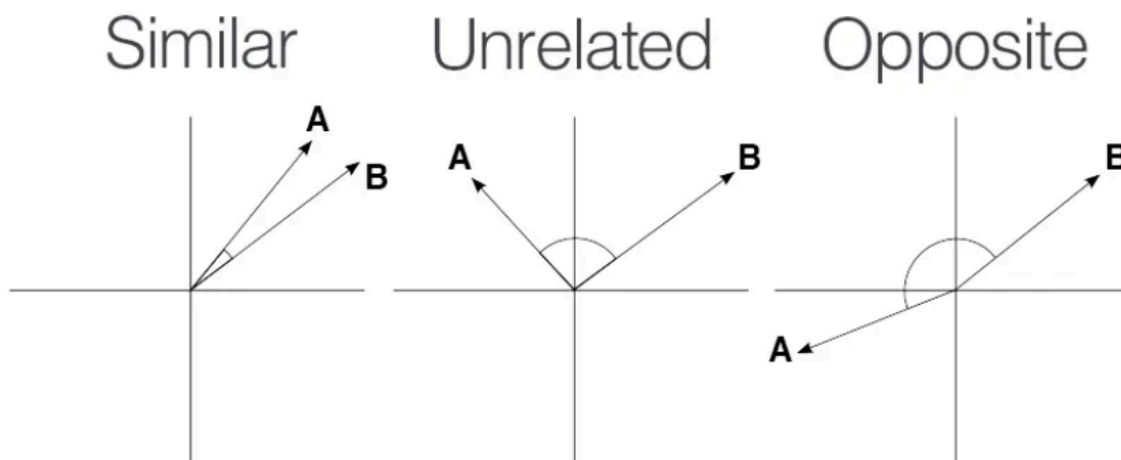
10.2.9 File Upload

To upload manuals, hover on the left side of the settings page to open the sidebar/history. At the bottom left, click the **file upload icon**. You’ll be asked for a password enter **1234** (Hardcoded). This will take you to the upload page, where you can add manuals for **AMM, IPC, or SRM**. When uploading, make sure you choose the correct manual type and include all documents for that type. For example, if you are uploading AMM manuals, upload every AMM document together.



10.3 Cosine Similarity

Cosine similarity is a measurement that quantifies the similarity between two vectors based on the cosine of the angle between them.



This metric is used to measure how similar two non-zero vectors are by calculating the cosine of the angle between them, ranging from -1 (opposite) to 1 (identical), with 0 indicating orthogonality (unrelated)

How the solution uses it:

- **Natural Language Processing (NLP):** Comparing document similarity based on word frequency or embeddings.
- **Semantic Search:** Finding documents that are contextually related to a query.